Computer Networks

Course Code: CSC263
Year/ Semester: II/IV

Compiled by Shishir Ghimire

Credit Hours: 3hrs

Unit - 05: Transport Layer

Class Load : 6 hrs

TABLE OF CONTENTS

Transport Layer (Revisit):

- Known as “Heart of OSI Model”.
- The Transport layer ensures that messages are transmitted in the order in which they are sent and there is no duplication of data.
- The main responsibility of the transport layer is to transfer the data completely.
- It receives the data from the upper layer and converts them into smaller units known as segments.
- This layer can be termed as an end-to-end layer as it provides a point-to-point connection between source and destination to deliver the data reliably.
- When the message reaches the transport layers, one transport layer protocols TCP or UDP is selected.
- Normally, OSI transport layer is connection oriented . i.e. TCP is selected.
- Data in the Transport Layer is called Segments.

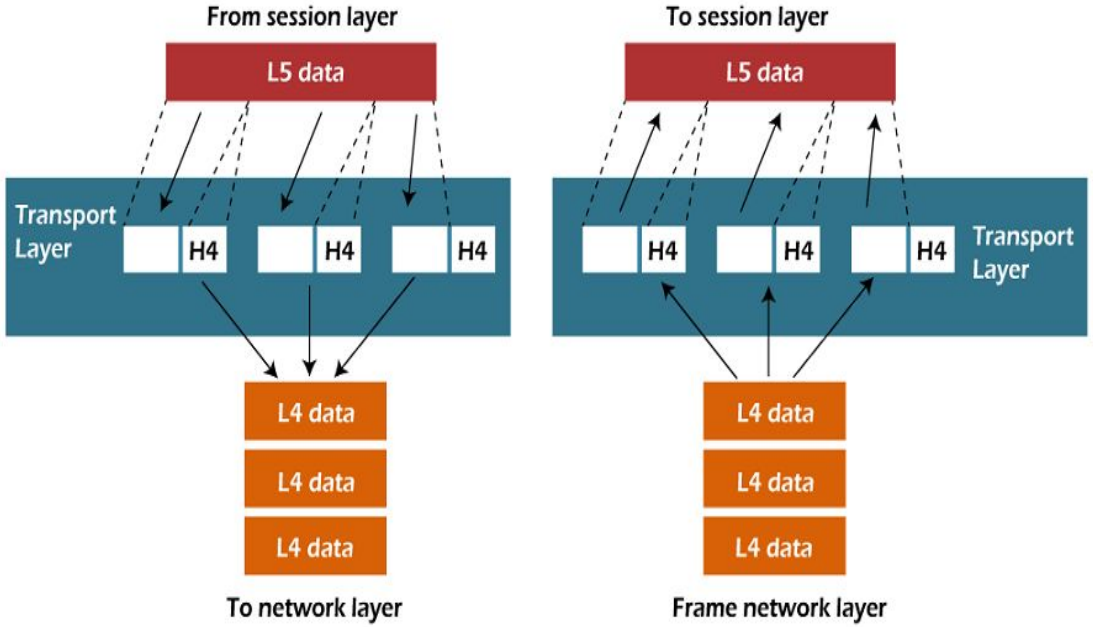
Transport Layer (Revisit):

TCP

- Supports segmentation
- Connection oriented
- Reliable
- TCP segment

UDP

- Does not support segmentation
- Connection less
- Unreliable
- UDP datagram



Transport Layer (Revisit):

Functions of the Transport Layer

- **Segmentation and Reassembly:** This layer accepts the message from the (session) layer, and breaks the message into smaller units. Each of the segments produced has a header associated with it. The transport layer at the destination station reassembles the message.
- **Service Point Addressing:** To deliver the message to the **correct process**, the transport layer header includes a type of address called service point address or port address. Thus by specifying this address, the transport layer makes sure that the message is delivered to the correct process.
- **Connection Control:** Transport layer provides two services connection-oriented service and connectionless service.

Introduction, Functions & Services

5.1

Introduction:

- Known as “Heart of OSI Model”.
- The Transport layer ensures that messages are transmitted in the order in which they are sent and there is no duplication of data.
- The main responsibility of the transport layer is to transfer the data completely.
- It receives the data from the upper layer and converts them into smaller units known as segments.
- This layer can be termed as an end-to-end layer as it provides a point-to-point connection between source and destination to deliver the data reliably.
- When the message reaches the transport layers, one transport layer protocols TCP or UDP is selected.
- Normally, OSI transport layer is connection oriented . i.e. TCP is selected.
- Data in the Transport Layer is called Segments.

Functions of Transport Layer:

- **Process to Process Delivery**
  - It refers to the **end-to-end** delivery of data between applications running on different devices within a network.
- **Multiplexing and Demultiplexing**
  - Multiplexing involves combining **multiple data streams** into a **single channe**l for transmission.
  - De-multiplexing is the process of **separating the combined signals** back into their original, individual components.
- **Segmentation and Reassembly**
  - **Breaking** a large message into smaller units called segments for efficient transmission.
  - **Reconstructing** the original message from the received segments.
- **Congestion Control**
  - Managing and preventing network congestion to ensure efficient data flow.

Functions of Transport Layer:

- **Connection Control – TCP or UDP**
  - ***TCP:*** A connection-oriented protocol that provides reliable, ordered delivery of data.
  - ***UDP:*** A connectionless protocol that provides faster but less reliable data transmission.
- **Flow Control**
  - Managing the **rate of data transmission** between sender and receiver to avoid congestion.
- **Error Control**
  - Ensuring data integrity by **detecting and correcting errors** that may occur during transmission.

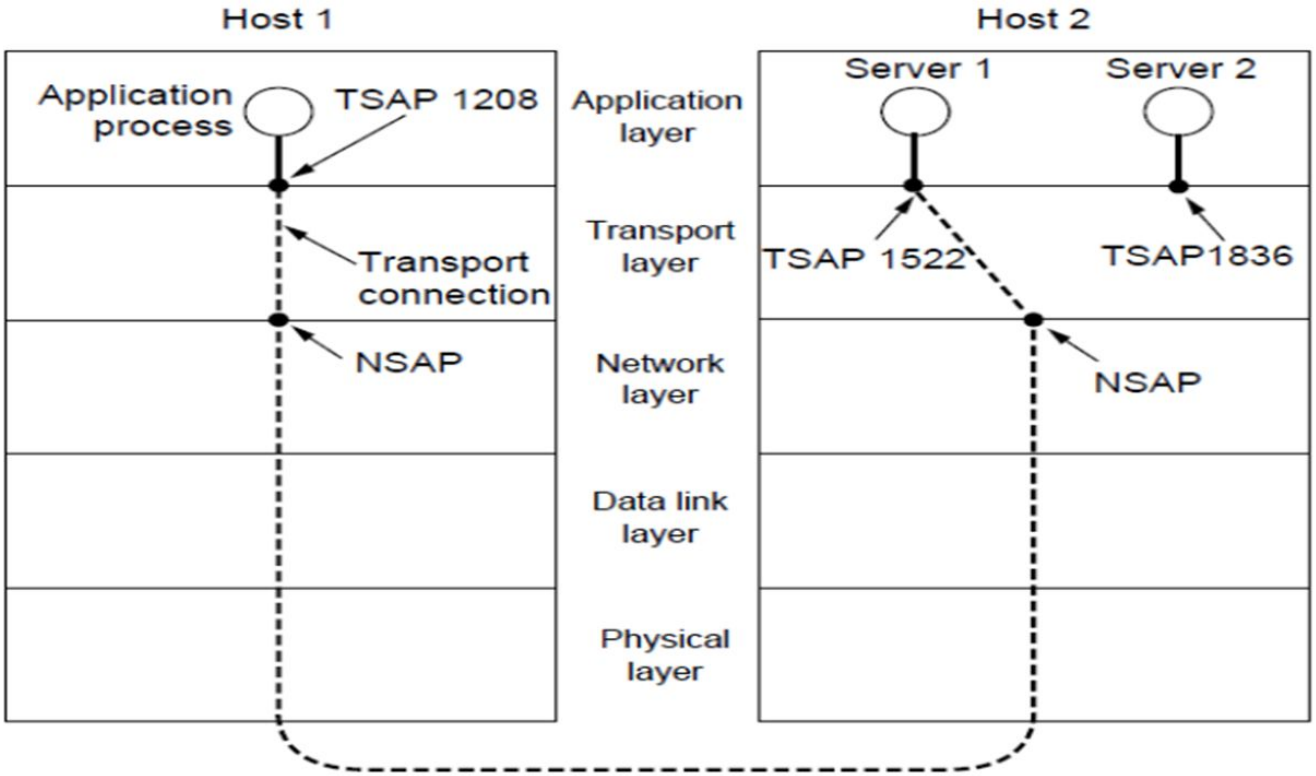
Elements of Transport Protocols:

1. Addressing
2. Establishing a connection
3. Releasing a connection
4. Flow control and buffering
5. Multiplexing
6. Crash recover

Addressing:

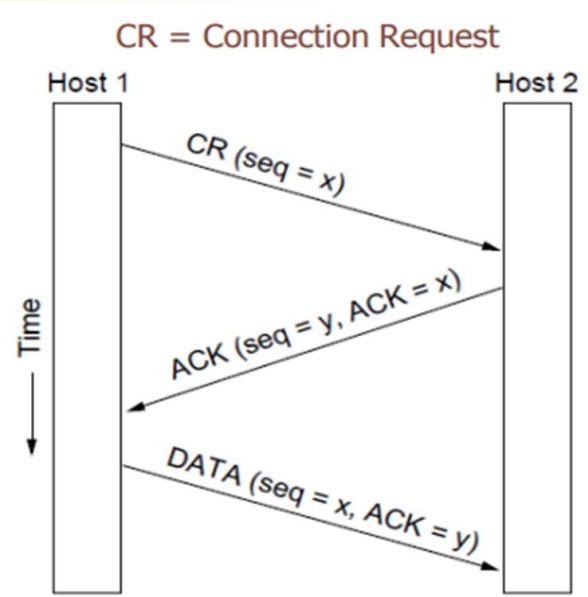
- When a **application process** want to setup a connection to a remote application process, it must **specify which host process** to connect to.
- Addressing is used to define **transport addresses** to which processes can **listen for connection requests.** In the context of internet these end points are called **ports.**
- **Application process** is connected to the TSAP.
- **Entity** connects to the NSAP.
- There are multiple processes running within the host.
- To access a specific service, we have to mention a **specific port address**.
  - **SAP:** Service Access Point
  - **TSAP :**Transport SAP
  - **NSAP :** Network SAP

Addressing:


Host 1
Host 2
Application process
TSAP 1208
Server 1
Server 2
Application layer
Transport layer
Transport connection
TSAP 1522
TSAP1836
Network layer
NSAP
NSAP
Data link layer
Physical layer

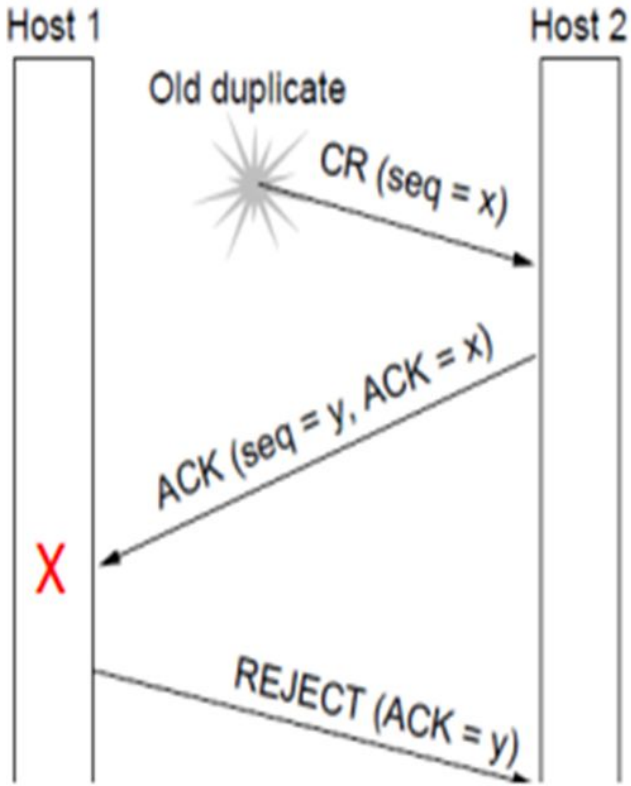
Connection Establishment:

- In **normal situation** Connection is being established by **3 way handshake**.
  - H1 chooses **Initial Sequence Number** (ISN) **x**
  - H2 receives the initial communication from H1 and **responds** to it.
  - H2 acknowledges the receipt of the initial sequence number x from H1.
  - H2 selects its **own** initial sequence number (ISN) for the communication. Let's say y is the chosen ISN.
  - H2 informs H1 about its selected ISN (y) as part of the **connection establishment**.
  - H1 acknowledges the receipt of H2's ISN (y).
  - H1 sends the first data segment to H2.



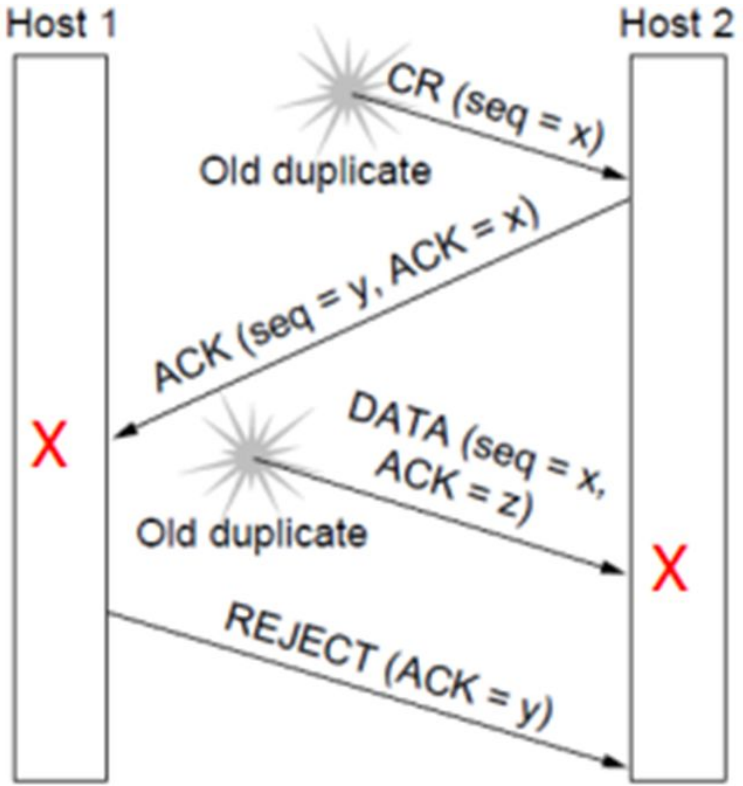
Abnormal Situation:

- Delayed **duplicate CR**
  - H2 sends ACK to H1
  - H1 rejects
  - H2 knows it was tricked.



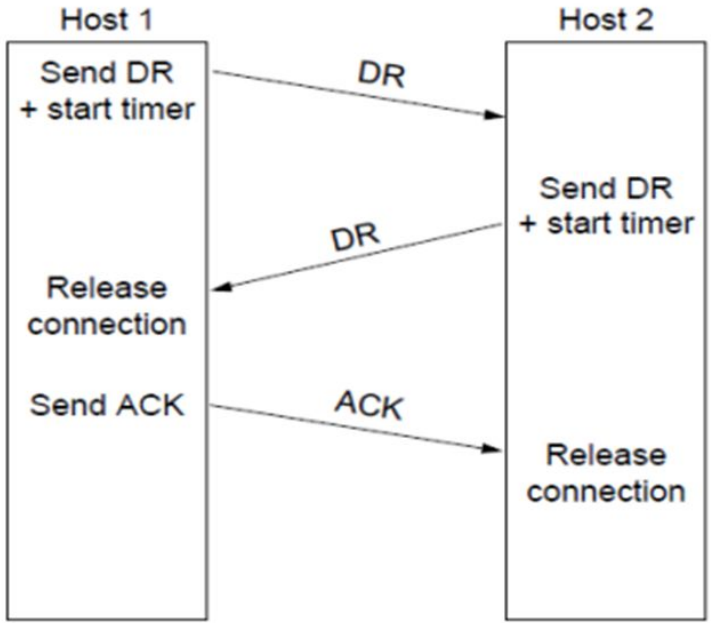
Abnormal Situation:

- **Worst case**
  - Old ACK floating
  - H2 gets delayed CR, replies
  - H1 rejects
  - H2 gets old DATA, discards (z received instead of y)



Connection Release:

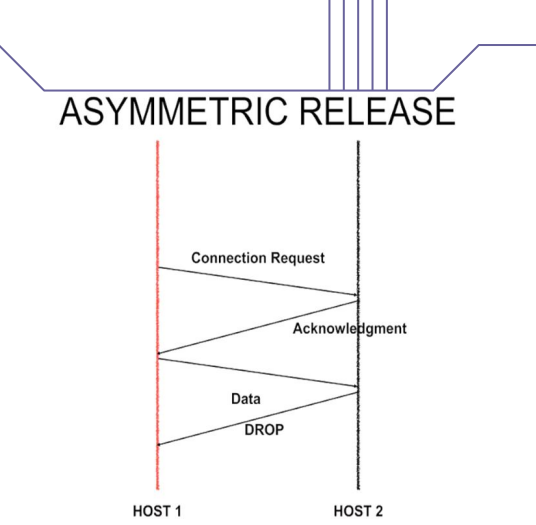
- Disconnection connection between two user.
- Normal **release sequence:**
  - H1 send DR (Disconnection Request), start timer
  - H2 responds with DR
  - when H1 receive DR, release
  - when H2 receive ACK, release

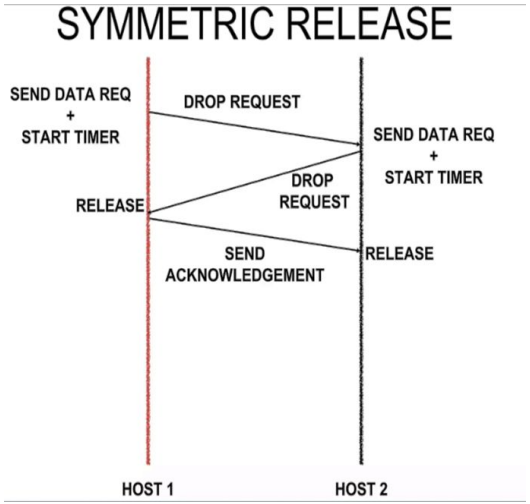


Connection Release:

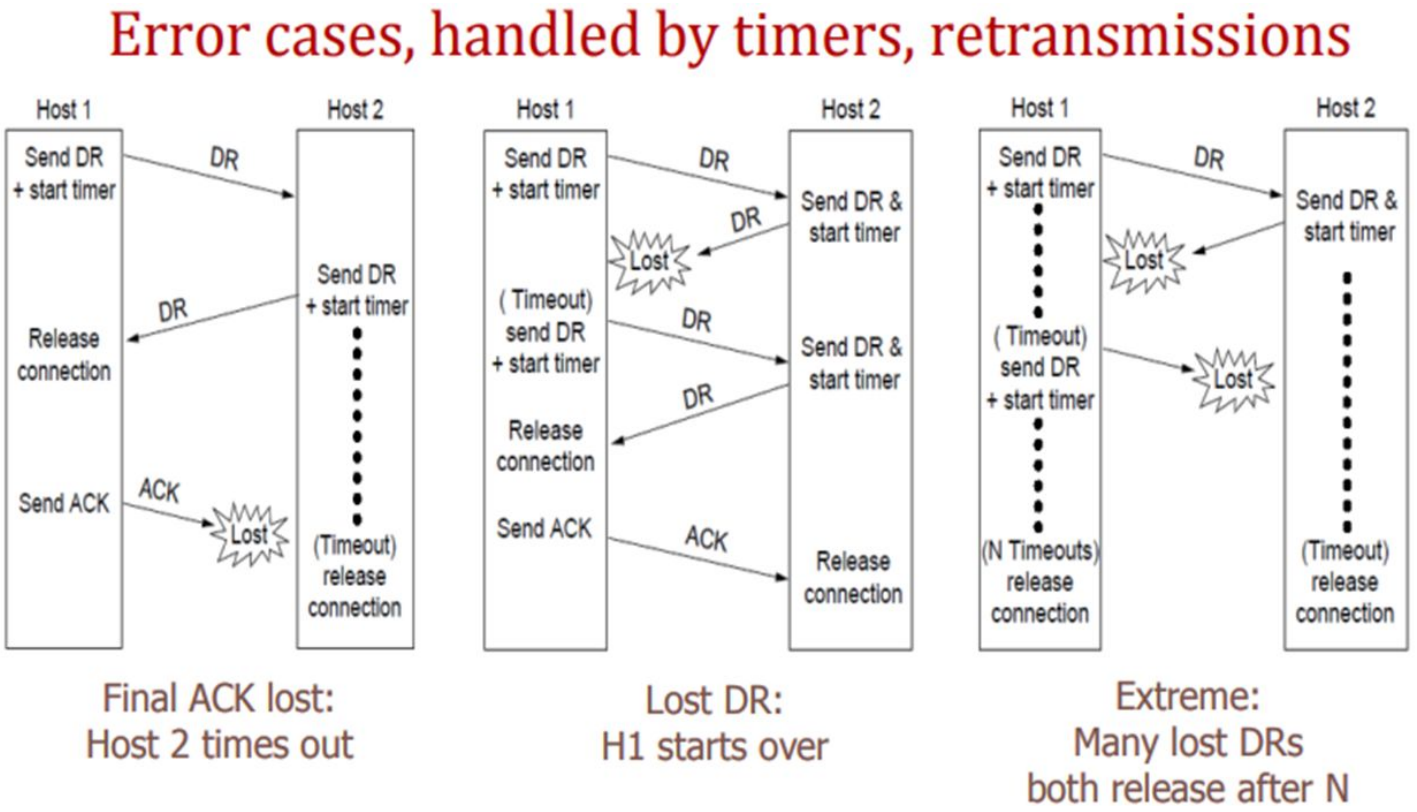
- **Asymmetric Release:**
  - ➢ when one party hangs up, the connection is broken.

- **Symmetric Release:**
  - ➢ Each direction is released independently.
  - ➢ Can receive data after sending DISCONNECT
  - ➢ H1: I am done, are you done too?
  - ➢ H2: I am done too, goodbye



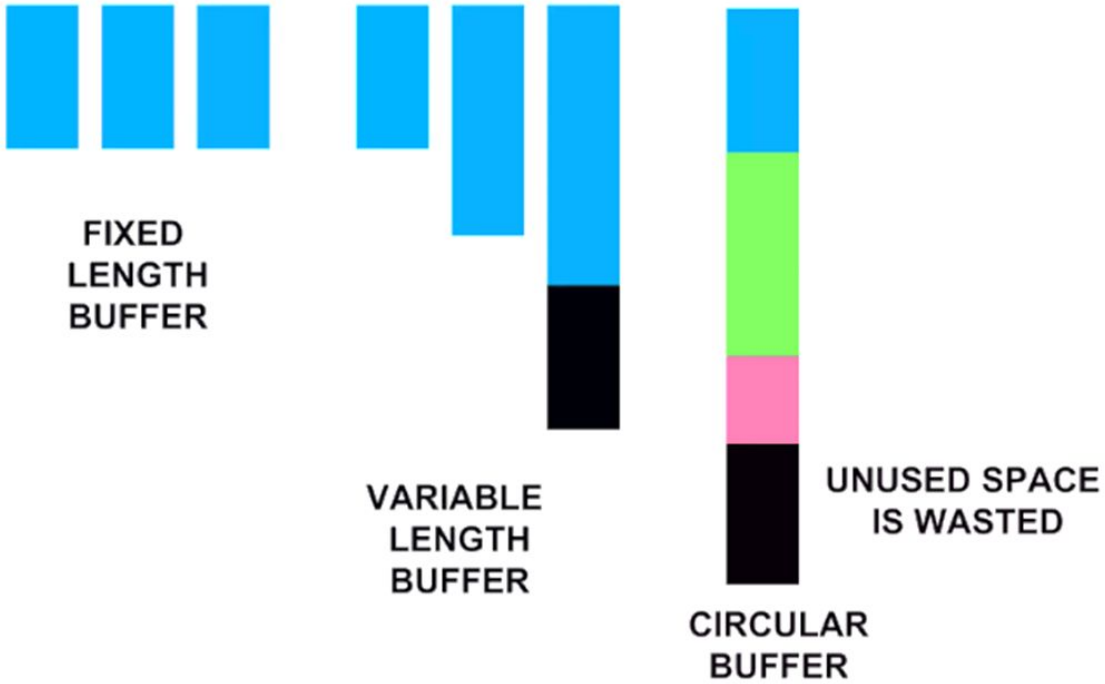


Connection Release:



Flow Control:

To control buffer ,transport layer manages **buffer.**

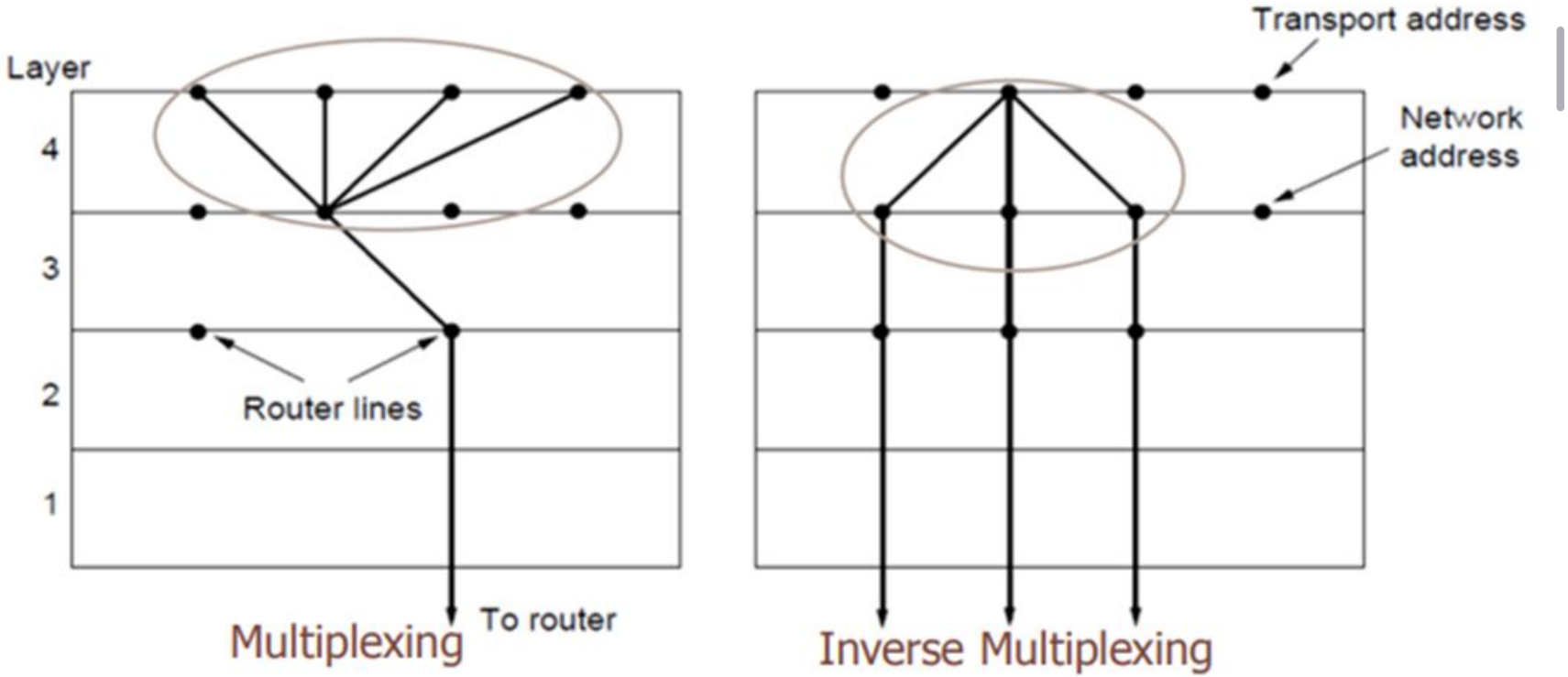


Multiplexing:

In Transport layer, network sharing can either be:

- **Multiplexing:**
  - Multiple connections **share a common** network address.
  - This is achieved by using **different identifiers** within the communication protocol. For example, in TCP/IP, multiplexing is often done using **port numbers.**
- **Inverse multiplexing:**
  - Multiple network addresses **share a common connection**, by **splitting** a high-speed connection into lower-speed connections.

Multiplexing:



Crash Recovery:

- It refers to the mechanism of recovery of network from crash.
  - If Router Crashes,
    - No problem
    - **Datagram Network:** Loss of packet is always handled
    - **Connection-oriented Network:** Establish new connection + use state to continue service.
  - If Host Crashes,
    - Recovery from N crashes is only done from N+1 layer and only if the higher layer **retains enough information**.

Transport Protocols: TCP, UDP and Their Comparisons

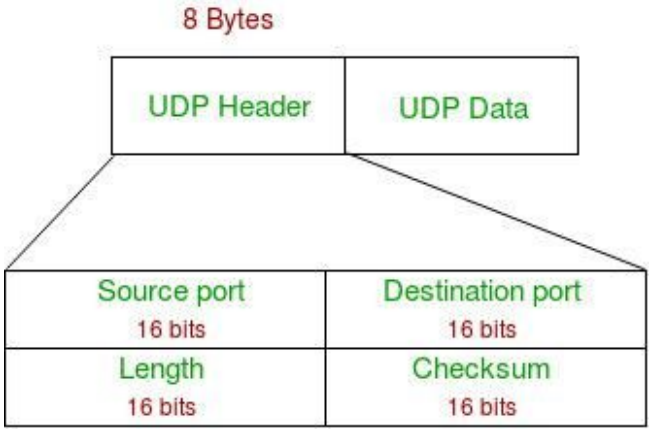
5.2

Overview:

- The Internet has **two main protocols** in the transport layer, a connectionless protocol and a connection-oriented one.
- Commonly used protocol in Transport Layer are:
  - TCP (Connection Oriented Protocol)
  - UDP (Connectionless Protocol)

User Datagram Protocol (UDP):

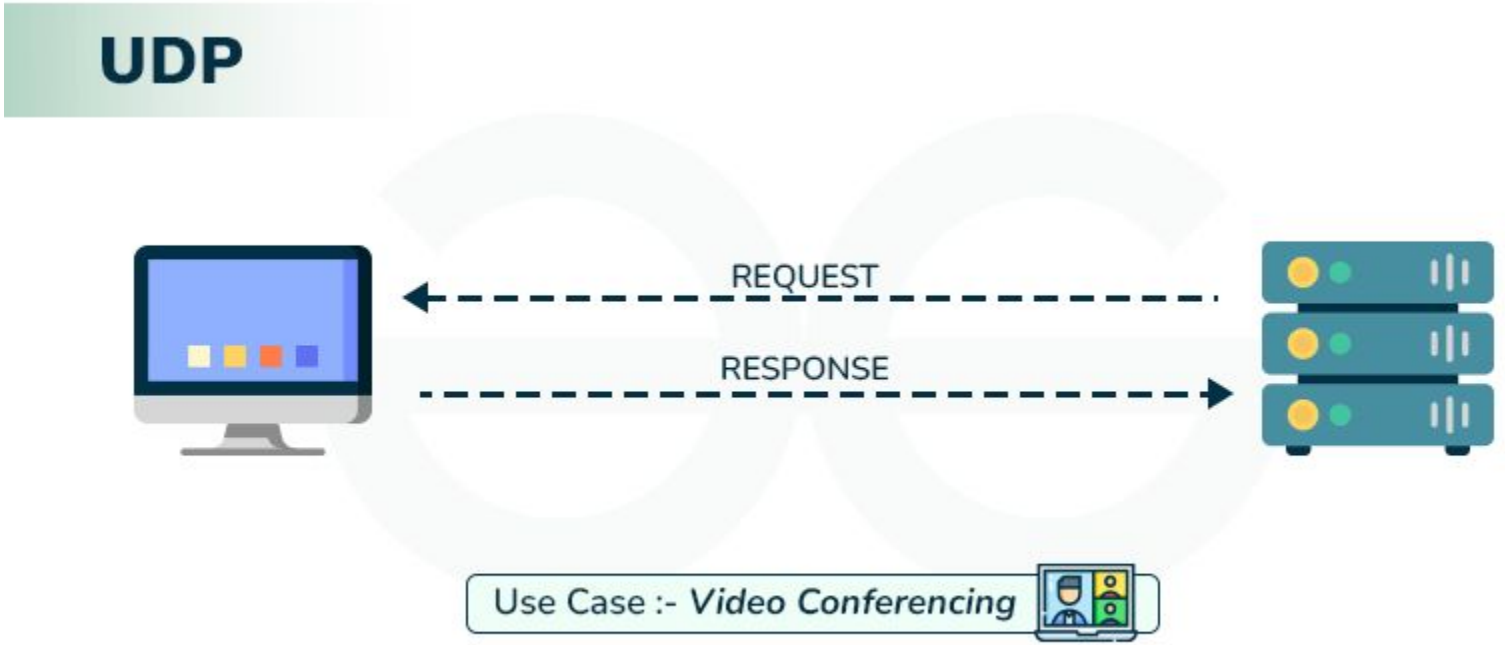
- The user datagram protocol (UDP) is called a **connectionless, unreliable** transport protocol.
- It does not add anything to the services of IP except to provide **process-to-process communication** instead of host-to-host communication.
- This type of protocol is used when reliability and security are **less important** than speed and size.
- The UDP packet structure is as follows:



User Datagram Protocol (UDP):

- **Source Port:** Source Port is a 2 Byte long field used to identify the **port number** of the source.
- **Destination Port:** It is a 2 Byte long field, used to identify the port of the **destined packet**.
- **Length:** Length is the length of UDP including the header and the data. It is a 16-bits field.
- **Checksum:** Checksum is 2 Bytes long field. It is the 16-bit one's complement of the one's complement sum of the UDP header.

User Datagram Protocol (UDP):



UDP Operations:

- **Connectionless Services:**
  - UDP provides connectionless service which means that each user datagram sent by UDP is an **independent datagram**.
  - There is **no relationship** between the different user datagrams even if they are coming from the **same source process** and going to the **same destination process**.
  - The user datagrams are **not numbered** and there is no connection **establishment** and no connection **termination**, which means each user datagram can travel on a different path. Stream of data cannot be sent; it should get fragmented.
- **Flow and Error Control:**
  - UDP is a very simple, unreliable transport protocol which **does not provide** flow control.
  - The receiver may overflow with incoming messages. There is no error control mechanism in UDP except checksum, which means the **sender does not know** if a message has been lost or duplicated.

UDP Operations:

- When the receiver detects an error through the checksum, the user datagram is silently **discarded**.

Encapsulation and Decapsulation:

- To send a message from one process to another, the UDP protocol **encapsulates and de-capsulates** messages in an IP datagram.

Queuing:

- Queuing in UDP simply refers to **requesting port number** for client processes and **using that port number** for process-to-process delivery of messages.

UDP Characteristics:

- UDP allows **very simple** data transmission without any error detection. So, it can be used in networking **applications like** VOIP, streaming media, etc. where **loss of few packets** can be tolerated and still function appropriately.
- UDP is **suitable for multicasting** since multicasting capability is embedded in UDP software but not in TCP software.
- UDP is used for **management processes** such as SNMP.
- UDP is used for some route updating protocols such as RIP.
- UDP is used by Dynamic Host Configuration Protocol (DHCP) to assign IP addresses to systems dynamically.
- UDP is an ideal protocol for network applications where **latency is critical** such as gaming and voice and video communications.

UDP Characteristics:

- **Other characteristics:**
  - Supports broadcasting, multicasting (not in TCP)
  - Packet oriented. (TCP gives byte stream)
  - Simple protocol.

- **UDP Application:**
  - Here are few applications where UDP is used to transmit data:
    - Domain Name Services
    - Simple Network Management Protocol
    - Trivial File Transfer Protocol
    - Routing Information Protocol
    - Kerberos

UDP Advantages/Disadvantages:

Advantages of UDP

- It does not require any connection for sending or receiving data.
- Broadcast and Multicast are available in UDP.
- UDP can operate on a large range of networks.
- UDP has live and real-time data.
- UDP can deliver data even if all the components of the data are not complete.

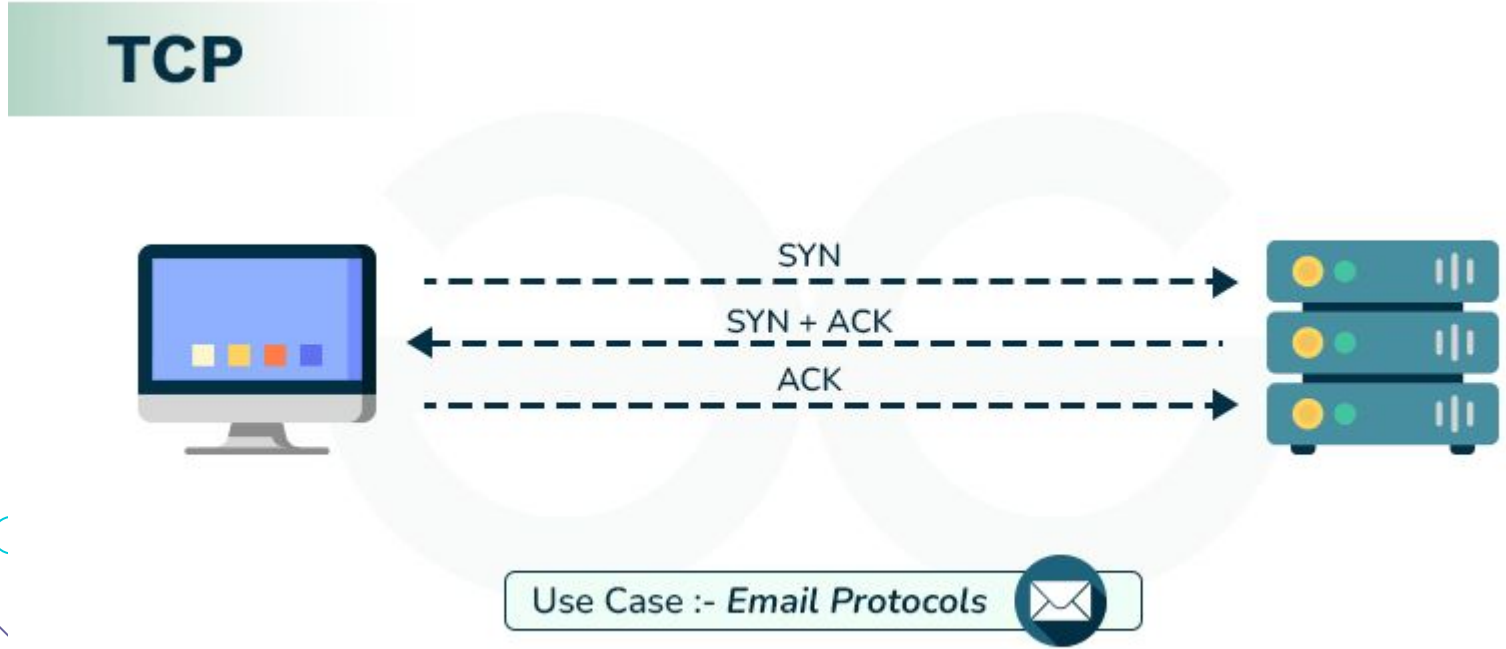
Disadvantages of UDP

- We don't have any way to acknowledge the successful transfer of data.
- UDP cannot have the mechanism to track the sequence of data.
- UDP is connectionless, and due to this, it is unreliable to transfer data.
- In case of a Collision or error detection, UDP packets are dropped by Routers in comparison to TCP.

Transmission Control Protocol (TCP):

- TCP stands for **Transmission Control Protocol**.
- It is a transport layer protocol that facilitates the **transmission of packets** from source to destination.
- It is a **connection-oriented protocol** that means it establishes the connection **prior** to the communication that occurs between the computing devices in a network.
- This protocol is used with an IP protocol, so together, they are referred to as a TCP/IP.

TCP(Transmission Control Protocol):

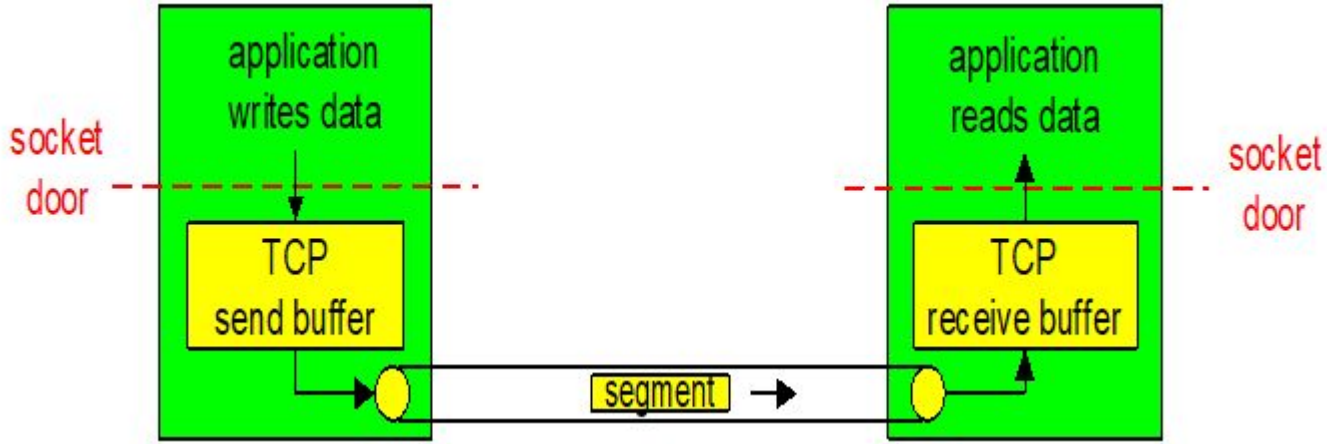


TCP(Transmission Control Protocol):

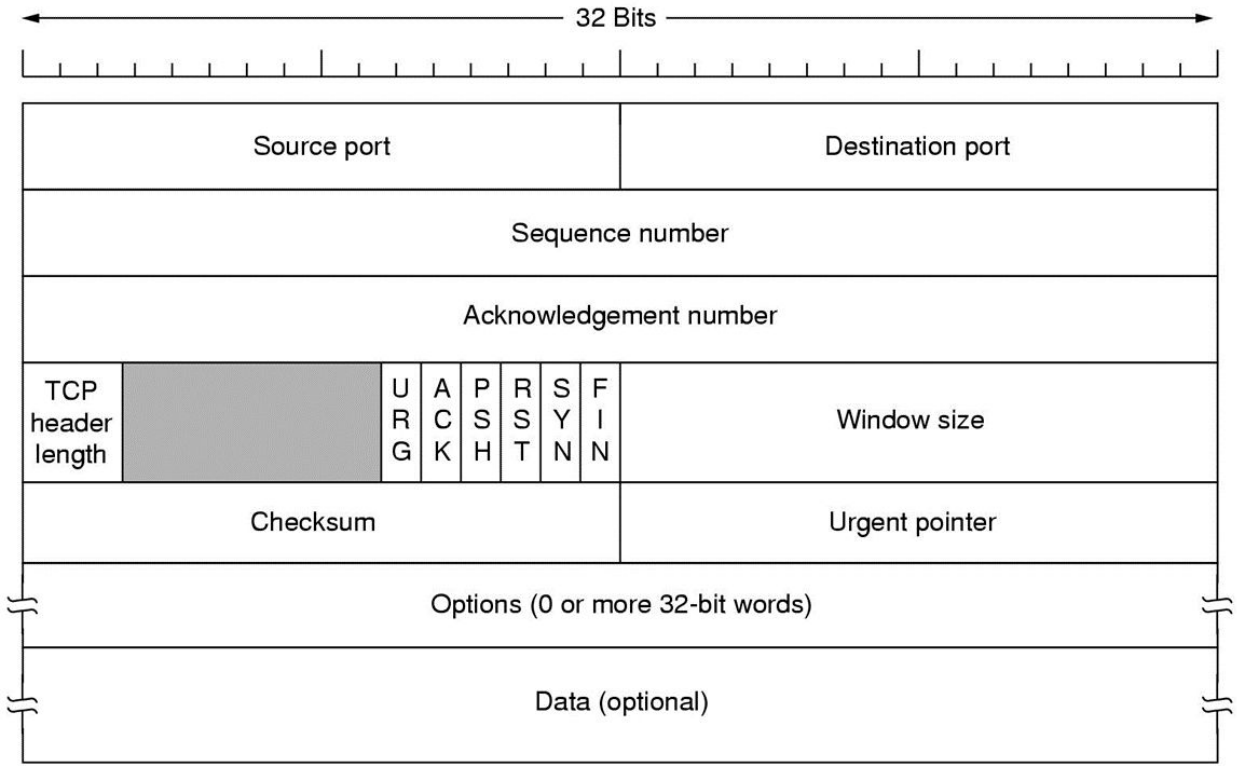
- **TCP services:**
  - **Point-to-Point**
    - one sender, one receiver
  - reliable, in-order byte stream
  - **Pipelined & Flow controlled**
    - window size set by TCP congestion and flow control algorithms
  - **Connection-Oriented**
    - handshaking to get at initial state
  - **Full duplex data**
    - bi-directional data flow in same connection

TCP(Transmission Control Protocol):

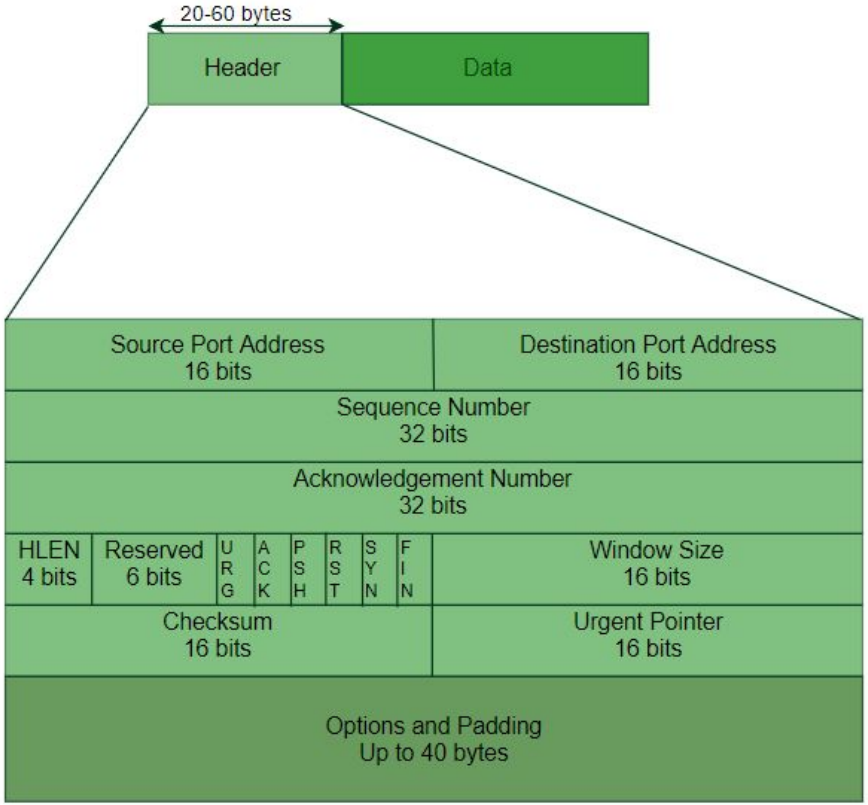
- send & receive buffers



TCP Packet Structure:



TCP Packet Header Format:



TCP Packet Structure:

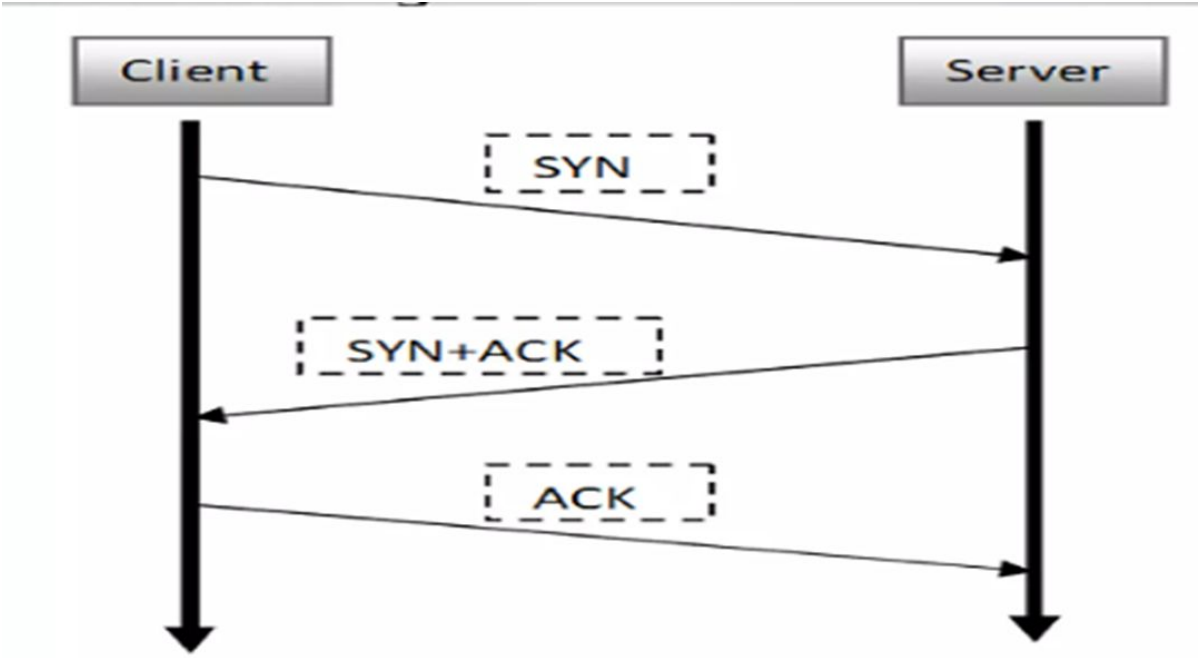
- **Source port:** It defines the port of the application, which is sending the data. So, this field contains the **source port address**, which is 16 bits.
- **Destination port:** It defines the port of the application on the receiving side. So, this field contains the **destination port address**, which is 16 bits.
- **Sequence number:** This field contains the **sequence number** of data bytes in a particular session. It is used to reassemble the message at receiving end if the segments are received out of order.
- **Acknowledgment number:** It is a 32 bit field that holds the acknowledgement number i.e the byte number that receiver **expects to receive next**. It is an acknowledgement for the previous bytes being **received successfully**.

TCP Packet Structure:

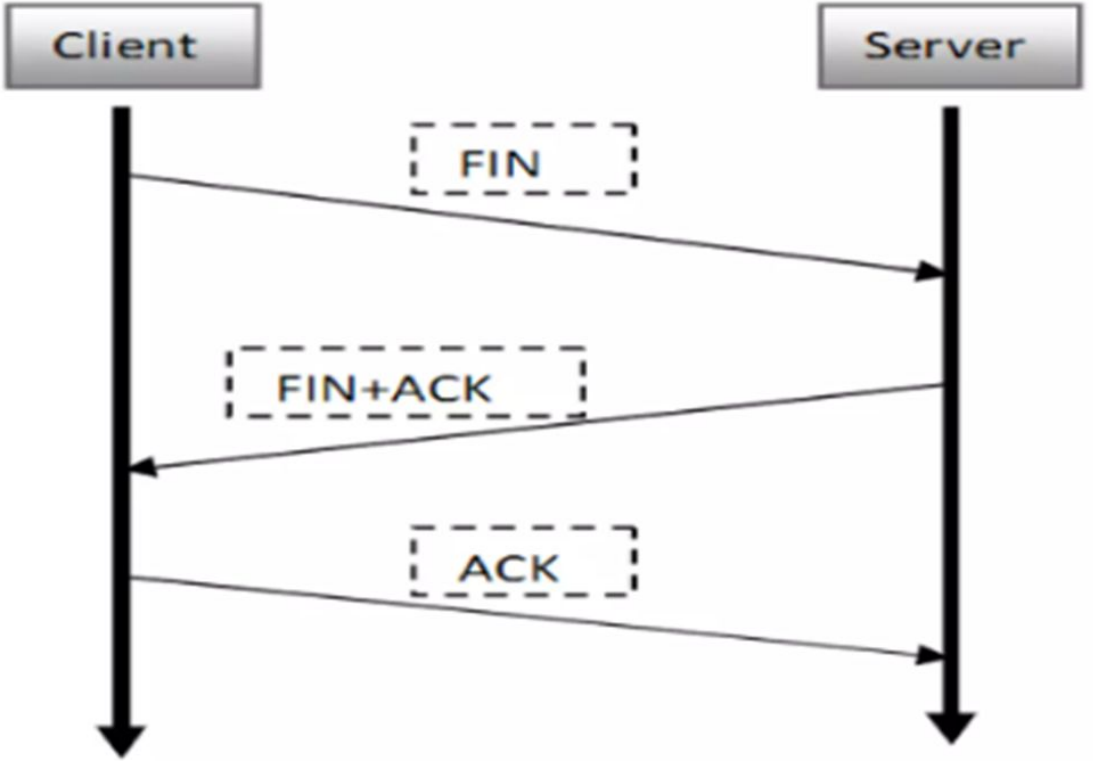
- **HLEN:** It specifies the **length of the header** indicated by the 4-byte words in the header. The size of the header lies between 20 and 60 bytes. Therefore, the value of this field would lie between 5 and 15.
- **Reserved:** It is a 4-bit field reserved for **future use**, and by default, all are set to zero.
- **Flags:**
  - **URG:** urgent pointer in use. If it is set, then the data is **processed urgently**.
  - **ACK:** Acknowledgement number is valid. If the ACK is set to 0, then it means that the data packet **does not contain** an acknowledgment.
  - **PSH:** If this field is set, then it requests the receiving device to **push the data** to the receiving application **without buffering** it.
  - **RST:** If it is set, then it requests to **restart a connection.**
  - **SYN:** It is used to **establish a connection** between the hosts.
  - **FIN:** It is used to **release/terminate a connection**, and no further data exchange will happen.

Connection establishment in TCP:

- Connection Establishment in TCP is established by three way handshaking.



Connection Release in TCP:



TCP Connection Management:

- TCP connection management can be represented using a **state diagram (FSD)**that illustrates the various states a TCP connection can be in and the transitions between these states.
- These **states are**:
  - Connection establishment
  - Data transfer
  - Connection termination
- i.e. to keep track of different all different events happening during connection.

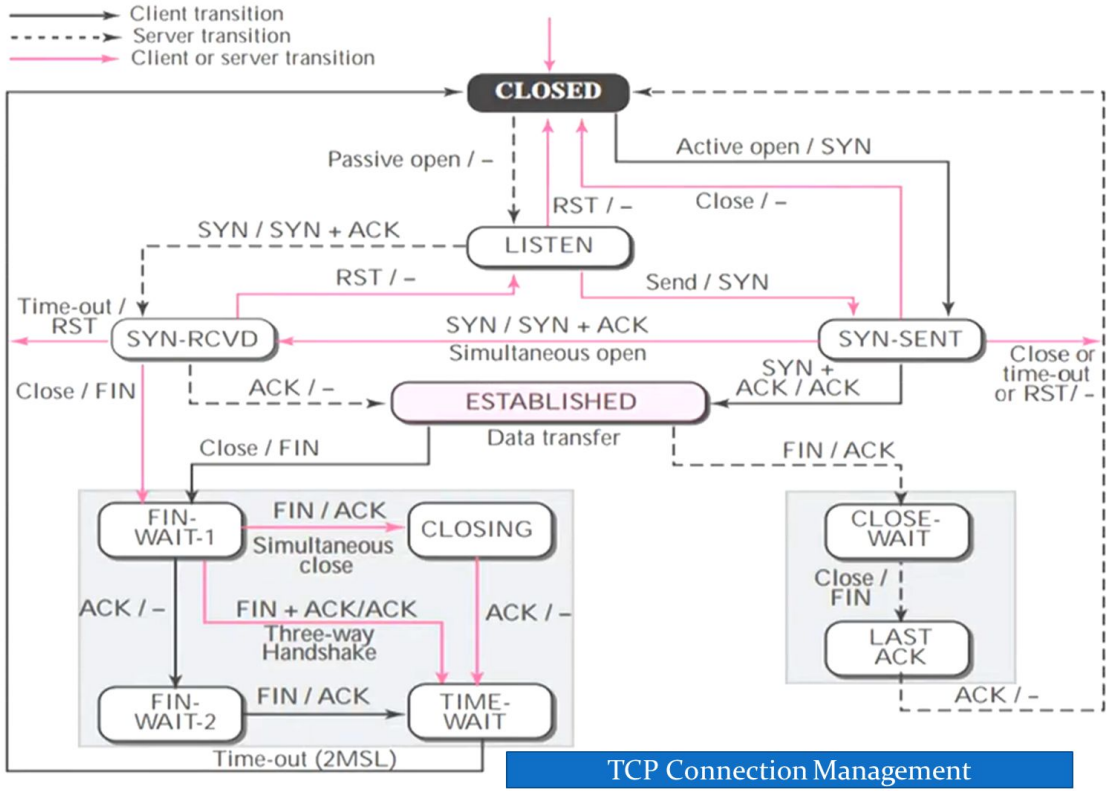
TCP Advantages/Disadvantages:

- **Advantages of TCP**
  - It is **reliable** for maintaining a connection between Sender and Receiver.
  - It is responsible for sending data in a **particular sequence**.
  - Its operations are not dependent on Operating System .
  - It allows and supports **many routing protocols.**
  - It can **reduce** the speed of data based on the speed of the receiver.
- **Disadvantages of TCP**
  - It is **slower** than UDP and it takes more bandwidth.
  - Slower upon starting of transfer of a file.
  - **Not suitable** for **LAN and PAN** Networks.
  - It **does not have** a multicast or broadcast category.
  - It does not load the whole page if a **single data** of the page is missing.

Application of TCP:

- **World Wide Web (WWW) :** When you browse websites, TCP **ensures reliable** data transfer between your browser and web servers.
- **Email :** TCP is used for sending and receiving emails. Protocols like SMTP (Simple Mail Transfer Protocol) handle email delivery across servers.
- **File Transfer Protocol (FTP) :** FTP relies on TCP to transfer **large files securely**. Whether you're uploading or downloading files, TCP ensures data integrity.
- **Secure Shell (SSH) :** SSH sessions, commonly used for remote administration, rely on **TCP for encrypted communication** between client and server.
- **Streaming Media :** Services like Netflix, YouTube, and Spotify use TCP to stream videos and music. It ensures **smooth playback** by managing data segments and retransmissions.

TCP Connection Management:



TCP Connection Management

TCP Connection Management:

State	Description
Closed	No connection is active or pending
Listen	The server is waiting for an incoming call
SYN rcvd	A connection request has arrived; wait for ACK
SYN sent	The application has started to open a connection
Established	The normal data transfer state
FIN wait 1	The application has said it is finished
FIN wait 2	The other side has agreed to release
Timed wait	Wait for all packets to die off
Closing	Both sides have tried to close simultaneously
Close wait	The other side has initiated a release
Last Ack	Wait for all packets to die off

TCP vs UDP :

	TCP	UDP
Full form	It stands for **Transmission Control Protocol**.	It stands for **User Datagram Protocol**.
Type of connection	It is a connection-oriented protocol, which means that the connection needs to be established before the data is transmitted over the network.	It is a connectionless protocol, which means that it sends the data without checking whether the system is ready to receive or not.
Reliable	TCP is a reliable protocol as it provides assurance for the delivery of data packets.	UDP is an unreliable protocol as it does not take the guarantee for the delivery of packets.
Speed	TCP is slower than UDP as it performs error checking, flow control, and provides assurance for the delivery of	UDP is faster than TCP as it does not guarantee the delivery of data packets.
Header size	The size of TCP is 20 bytes.	The size of the UDP is 8 bytes.
Acknowledgment	TCP uses the three-way-handshake concept. In this concept, if the sender receives the ACK, then the sender will send the data. TCP also has the ability to resend the lost data.	UDP does not wait for any acknowledgment; it just sends the data.
Flow control mechanism	It follows the flow control mechanism in which too many packets cannot be sent to the receiver at the same time.	This protocol follows no such mechanism.
Error checking	TCP performs error checking by using a checksum. When the data is corrected, then the data is retransmitted to the receiver.	It does not perform any error checking, and also does not resend the lost data packets.
Applications	This protocol is mainly used where a secure and reliable communication process is required, like military services, web browsing, and e-mail.	This protocol is used where fast communication is required and does not care about the reliability like VoIP, game streaming, video and music streaming, etc.

Connection Oriented and Connectionless Services

5.3

Connection–Oriented Service:

Definition: A network service designed to establish a reliable end-to-end connection before data transmission.

Key Characteristics:

- Orderly Packet Delivery: Packets arrive in the order they were sent.
- Handshake Method: Establishes connection using a handshake process.
- Reliability: Known as a reliable network service due to its structured communication.

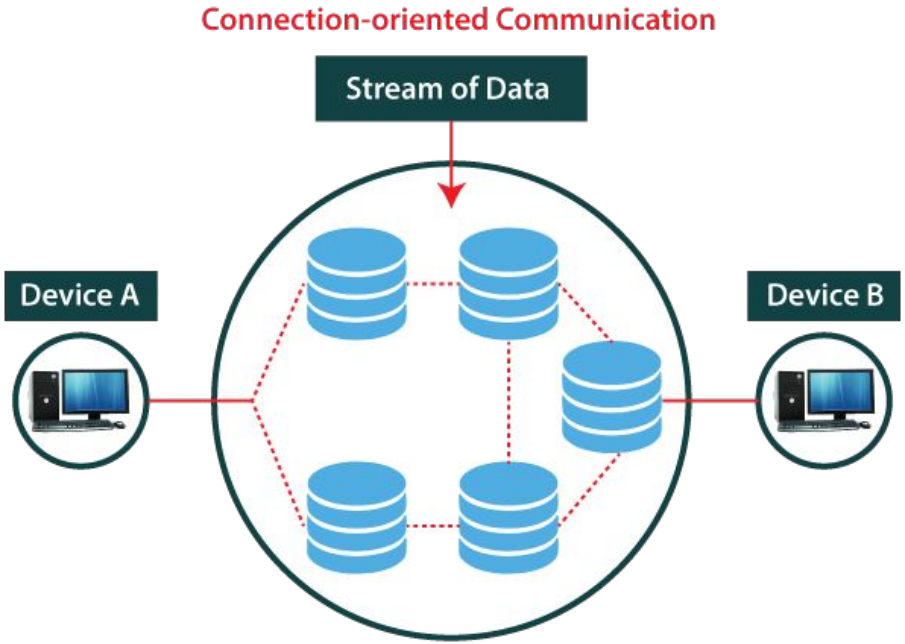
Communication Process:

1. **Connection Setup:**
   - SYN Packet: Sender sends a request to the receiver.
   - SYN-ACK Packet: Receiver responds to confirm readiness for communication.
2. **Data Transfer:**
   - Data packets are sent and received in the established order.
   - Both sender and receiver can send and receive data packets.
3. **Connection Termination:**
   - Sender terminates the connection after data exchange by sending a termination signal.

Connection–Oriented Service:

- Suppose, a sender wants to send data to the receiver. Then, first, the sender sends a request packet to a receiver in the form of an SYN packet.
- After that, the receiver responds to the sender's request with an (SYN-ACK) signal/packets.
- That represents the confirmation is received by the receiver to start the communication between the sender and the receiver.
- Now a sender can send the message or data to the receiver.
- Similarly, a receiver can respond or send the data to the sender in the form of packets.
- After successfully exchanging or transmitting data, a sender can terminate the connection by sending a signal to the receiver. In this way, we can say that it is a **reliable network service.**
- **Example Protocol: TCP**

Connection–oriented Service:



Connectionless Service:

Definition: A network service that does not establish a dedicated end-to-end connection before transmitting data.

Key Characteristics:

- No Setup Phase: Data packets are sent independently without a prior handshake.
- Packet Delivery: Packets may arrive out of order, and some packets may be lost.
- Best-Effort Delivery: Offers no guarantee of delivery, order, or duplicate protection.
- Lower Overhead: Faster for small amounts of data as it avoids connection setup and termination overhead.

Communication Process

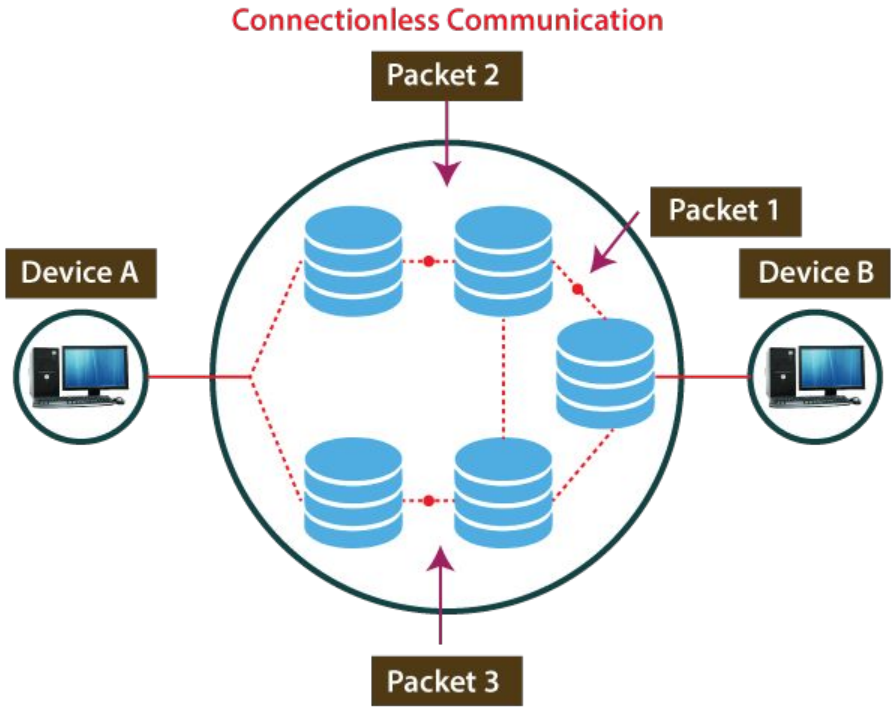
1. **Data Transmission:**
    - Each packet contains the destination address and is routed independently.
    - No acknowledgment of packet receipt is required.
2. **Packet Handling:**
    - Packets may take different paths to the destination.
    - There is no guarantee that packets will arrive in the correct order or even arrive at all.
3. **No Connection Termination:**
    - Since there is no established connection, termination is not necessary.

Connectionless Service:

- For example, a sender can directly send any data to the receiver without establishing any connection because it is a connectionless service.
- Data sent by the sender will be in the packet or data streams containing the receiver's address. In connectionless service, the data can be travelled and received in any order.
- However, it does not guarantee to transfer of the packets to the right destination. In this way, we can say that it is a **unreliable network service**.

- **Example Protocol: UDP**

Connectionless Service:



Connectionless

VS

Connection–oriented Network Service:

S. No	Comparison Parameter	Connection-oriented Service	Connection Less Service
1.	Related System	It is designed and developed based on the telephone system.	It is service based on the postal system.
2.	Definition	It is used to create an end to end connection between the senders to the receiver before transmitting the data over the same or different network.	It is used to transfer the data packets between senders to the receiver without creating any connection.
3.	Virtual path	It creates a virtual path between the sender and the receiver.	It does not create any virtual connection or path between the sender and the receiver.
4.	Authentication	It requires authentication before transmitting the data packets to the receiver.	It does not require authentication before transferring data packets.
5.	Data Packets Path	All data packets are received in the same order as those sent by the sender.	Not all data packets are received in the same order as those sent by the sender.
6.	Bandwidth Requirement	It requires a higher bandwidth to transfer the data packets.	It requires low bandwidth to transfer the data packets.
7.	Data Reliability	It is a more reliable connection service because it guarantees data packets transfer from one end to the other end with a connection.	It is not a reliable connection service because it does not guarantee the transfer of data packets from one end to another for establishing a connection.
8.	Congestion	There is no congestion as it provides an end-to-end connection between sender and receiver during transmission of data.	There may be congestion due to not providing an end-to-end connection between the source and receiver to transmit of data packets.
9.	Examples	Transmission Control Protocol (TCP) is an example of a connection-oriented service.	User Datagram Protocol (UDP), Internet Protocol (IP), and Internet Control Message Protocol (ICMP) are examples of connectionless service.

Congestion Control: Open Loop & Closed Loop, TCP Congestion Control

5.4

Congestion Control:

- It refers to the **techniques and mechanisms** used to **prevent** network congestion, which occurs when there is **more data** being transmitted through a network **than it can handle** efficiently.
- Congestion refers to a condition where the **demand for resources exceeds** the available capacity.
- Congestion occurs when there is more data **trying to traverse** a network than the network can handle efficiently.
- Congestion can lead to **degraded** performance, packet **loss**, and **increased** latency.

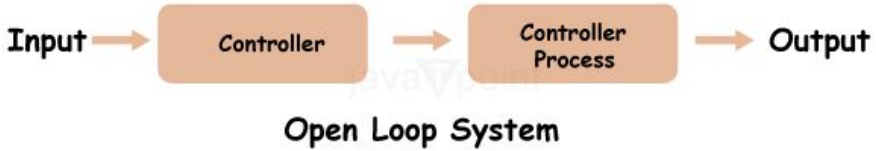
General Principles of Congestion Control

- The General Principles of Congestion Control are as follows:
  - **Open Loop Principle:**
    - Attempt to prevent congestion from **before it happens**.
    - After system is running, **no corrections made**.
    - The congestion is handled **either by** the source or destination.
  - **Closed Loop Principle:**
    - **monitor** system to detect congestion
    - pass information to where **action** is taken
    - **adjust system** operation to correct problem

General Principles of Congestion Control

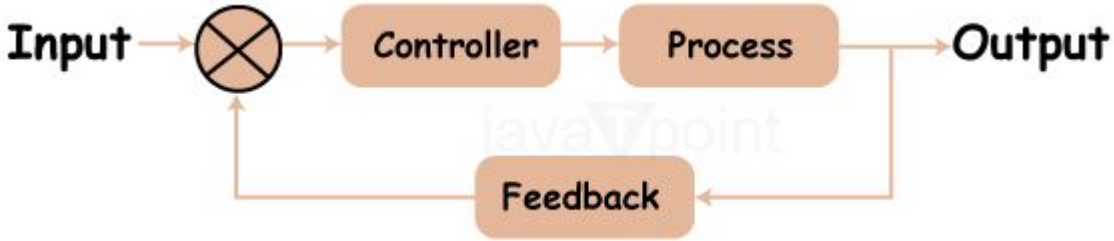
- The General Principles of Congestion Control are as follows:

  - **Open Loop Principle:**

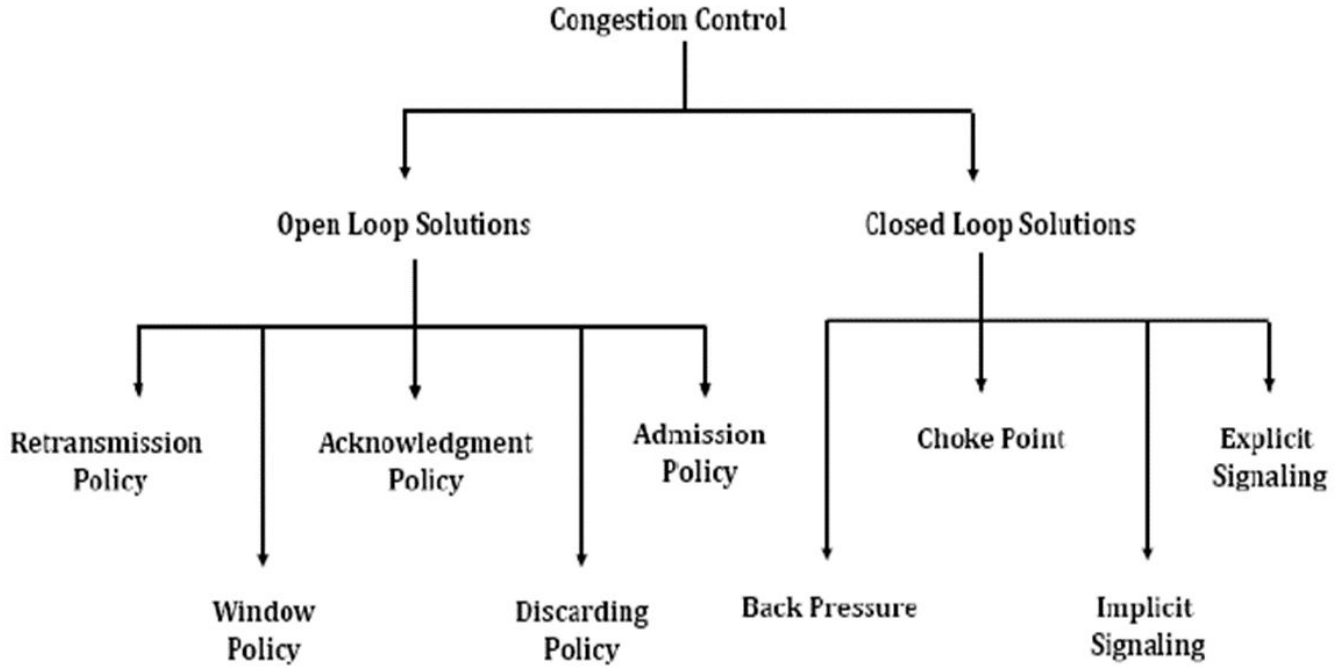


Open Loop System

  - **Closed Loop Principle:**



Principle of congestion control :



Open Loop Congestion Control:

- In open-loop congestion control, policies are applied to prevent congestion **before it happens.**
- In these mechanisms, congestion control is **handled by** either the source or the destination.

The list of policies that can prevent congestion are:

- **Retransmission Policy:**
  - It is the policy in which **retransmission of the packets** are taken care.
  - If the sender feels that a sent packet is lost or corrupted, the packet needs to be retransmitted. This transmission **may increase the congestion** in the network.
  - To prevent congestion, **retransmission timers** must be designed to prevent congestion and also able to optimize efficiency.

Open Loop Congestion Control:

- **Window Policy**
  - The type of window on sender side may also affect the congestion.
  - **Selective Repeat** is used for this policy to prevent congestion.
- **Acknowledgement Policy**
  - Since acknowledgement are also the **part of the load** in network, the acknowledgment policy imposed by the receiver **may also affect congestion.**
  - Several approaches can be used to prevent congestion related to acknowledgment.
  - The receiver should send acknowledgement for **N packets** rather than sending acknowledgement for a single packet.
  - The receiver should send an acknowledgment **only if** it has to send a packet or a timer expires.

Open Loop Congestion Control:

- **Discarding Policy**
  - A good discarding policy adopted by the routers is that the routers may prevent congestion and at the same time partially **discards the corrupted or less sensitive package** and also able to maintain the quality of a message.
  - In case of **audio** file transmission, routers can **discard fewer sensitive packets** to prevent congestion and also maintain the quality of the audio file.
- **Admission Policy**
  - Admission policy is a mechanism should be used to prevent congestion in a virtual circuit network.
  - Switches in a flow should first check the resource requirement of a network flow **before transmitting** it further.
  - If there is a chance of a congestion or there is a congestion in the network, router should **deny establishing a virtual network** connection to prevent further congestion.

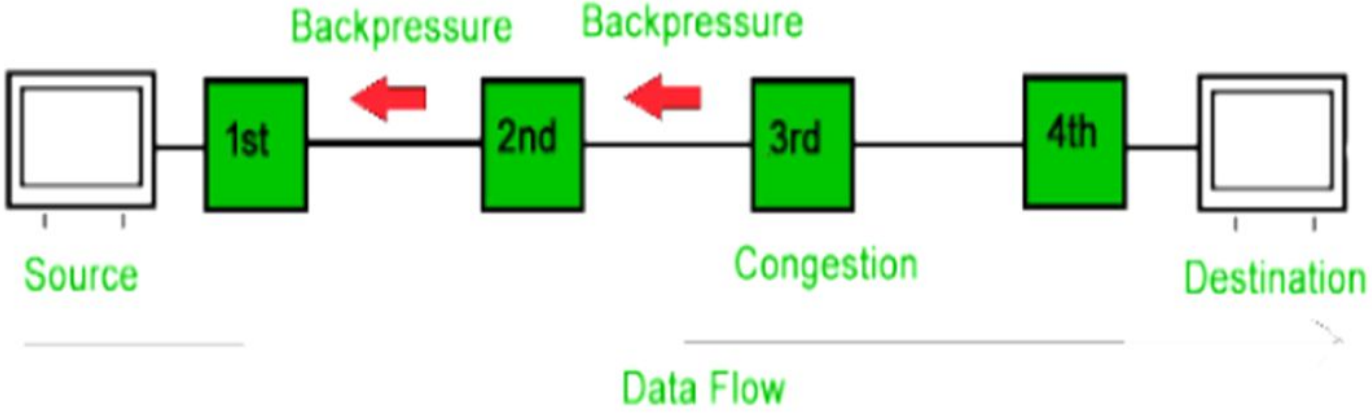
Closed Loop Congestion Control:

Closed-Loop congestion control mechanisms try to **reduce effects** of congestion **after** it happens.

- **Back Pressure:**
  - Backpressure is a technique in which a **congested node stops receiving packet** from upstream node.
  - This may cause the upstream node or nodes to become congested and **rejects receiving data** from above nodes.
  - Backpressure is a **node-to-node congestion control technique** that propagate in the **opposite direction** of data flow. The backpressure technique can be applied **only to** virtual circuit where each node has information of its above upstream node.

Closed Loop Congestion Control:

- In diagram below the **3rd node is congested** and **stops receiving packets** as a result 2nd node may be get congested due to slowing down of the output data flow. Similarly, 1st node may get congested and **informs the source** to slow down.

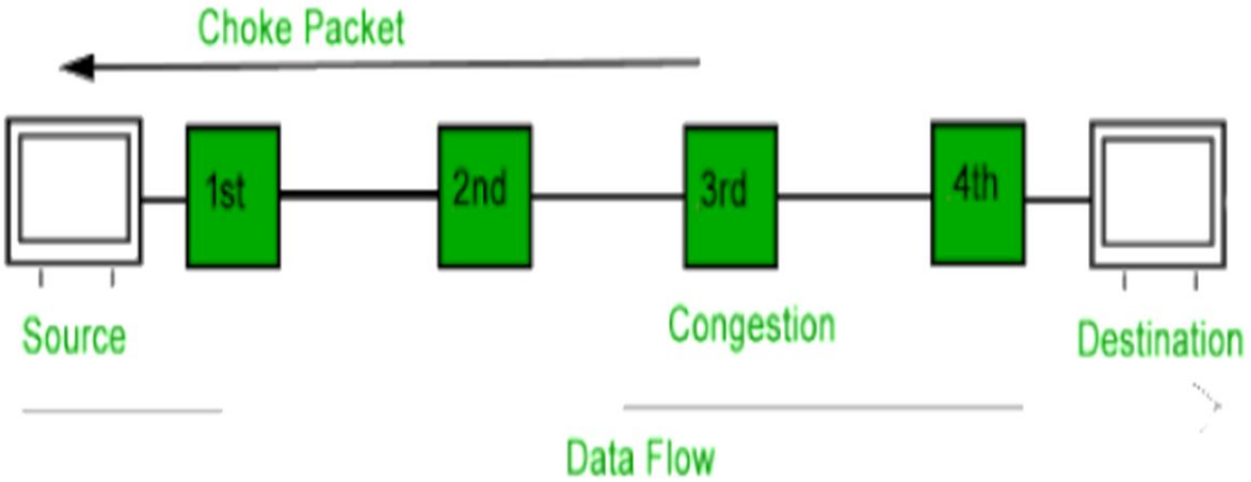


Closed Loop Congestion Control:

- **Choke Packet:**
  - Choke packet technique is **applicable to both** virtual networks as well as datagram subnets. A **choke packet** is a packet sent by a node to the source **to inform** it of congestion.
  - Each router **monitors** its resources and the utilization at each of its output lines. whenever the resource utilization **exceeds the threshold** value which is set by the administrator, the **router directly** sends a choke packet to the source giving it a **feedback to reduce** the traffic.
  - The intermediate nodes through which the packets have travelled are **not warned** about congestion.

Closed Loop Congestion Control:

- **Choke Packet**:



Closed Loop Congestion Control:

- **Implicit Signaling:**
  - In this method, there is **no communication between** congested node or nodes and the source.
  - The **source guesses** that there is congestion **somewhere in the network** from other symptoms.
  - **Example:** when source sends several packets and there is **no acknowledgment** for a while, one **assumption** is that network is congested and source should slow down.

Closed Loop Congestion Control:

- **Explicit Signaling:**
  - In explicit signaling, if a **node experiences** congestion it can **explicitly sends a packet** to the source or destination to inform about congestion.
  - The difference between choke packet and explicit signaling is that the **signal is included in the packets that carry data** rather than creating different packet as in case of choke packet technique.
    - **Forward Signaling :** In forward signaling, a **signal** is sent **in the direction** of the congestion. The **destination is warned** about congestion. The receiver in this case adopt policies to prevent further congestion.
    - **Backward Signaling :** In backward signaling, a signal is sent in the **opposite direction** of the congestion. The **source is warned** about congestion and it needs to slow down.

Traffic Shaping Algorithms: Leaky Bucket & Token Bucket

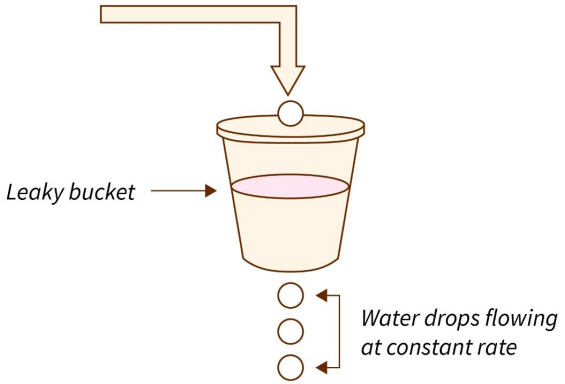
5.5

Traffic Shaping:

- It is a **mechanism to control** the **amount** and the **rate** of the traffic sent to the network.
- It means **regulating the average rate** flow of data to the network to prevent congestion.
- It maintains a **constant traffic pattern** that is sent to the network. This constant rate can depend on the network **bandwidth limits** and the **priority** of packets sent by the host.
- **Two techniques** can shape traffic:
  - Leaky bucket Algorithms
  - Token bucket Algorithms

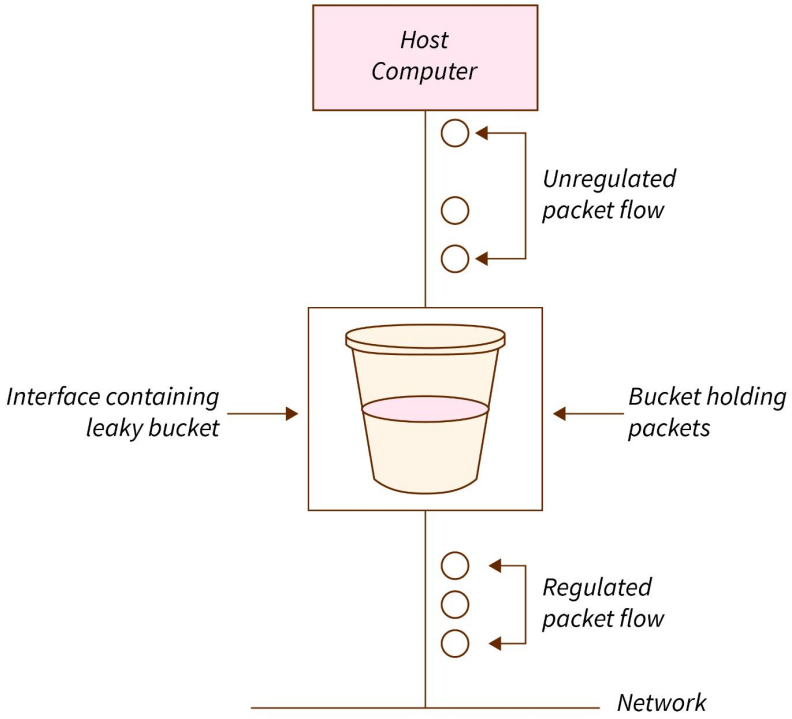
Leaky Bucket Algorithm:

- The leaky bucket algorithm is a method of congestion control where multiple packets are **stored temporarily**.
- These packets are sent to the network **at a constant rate** that is decided between the sender and the network.
- This algorithm is used to implement **congestion control** through **traffic shaping** in data networks.

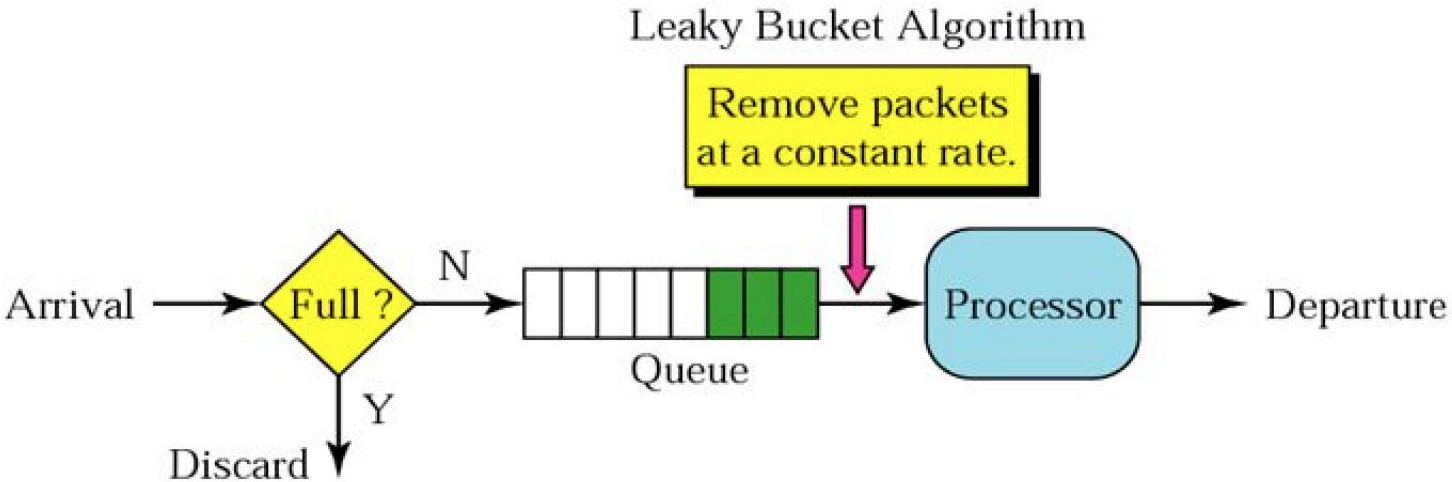


Leaky Bucket Algorithm:

- It is implemented as a **single server queue** with constant service time.
- If the bucket (Buffer) overflows then packets are discarded.
- In this algorithm the **input rate can vary** but the **output** remains **constant**.
- This algorithms saves busty traffic into **fixed rate traffic** by averaging the data rate.



Leaky Bucket Algorithm:



Leaky Bucket Algorithm > Implementation:

The leaky bucket algorithm can be implemented using a **FIFO (First In First Out) queue.** Packets that arrive first in the bucket should be transmitted first.

- A **queue** acts as a **bucket or a buffer** to hold the packets. This is implemented in the host operating system.
- Packets from the host are **pushed into the queue** as they arrive.
- At **some intervals**, the packets are sent to the network depending upon the **leak rate**. Generally, this interval is a clock tick. A clock tick is an interrupt generated from the physical clock to the processor.
- This leak rate is **predetermined** by the network. A network will guarantee some dedicated bandwidth for each host. This dedicated bandwidth can be used to set up this leak rate.
- If this **queue is full,** then the packets that arrive will be **discarded**.

Algorithm:

- **Step - 1 :** Initialize **buffer size** and **leak rate**.
- **Step - 2 :** At clock tick, Initialize **n** as the leak rate.
- **Step - 3 :** If the n is greater than the size of the packet at front of the queue, **send the packet** into the network.
- **Step - 4 :** Decrement **n by the size** of the packet.
- **Step - 5 :** Repeat Step 3 and Step 4 until **n is less** than the size of the packet at the front of the queue or **the bucket is empty**. No other packet can be transmitted till the next clock tick.
- **Step - 6 :** Go to Step 2.

Example:

Example

Let **n** = 1000

Packet =.

200	700	500	450	400	200

Since n > front of Queue i.e. n>200

Therefore, n= 1000-200 = 800

Packet size of 200 is sent to the network

200	700	500	450	400

Now Again n > front of queue i.e. n > 400

Therefore, n= 800-400 = 400

Packet size of 400 is sent to the network

200	700	500	450

Since n < front of queue .

There fore, the procedure is stop.

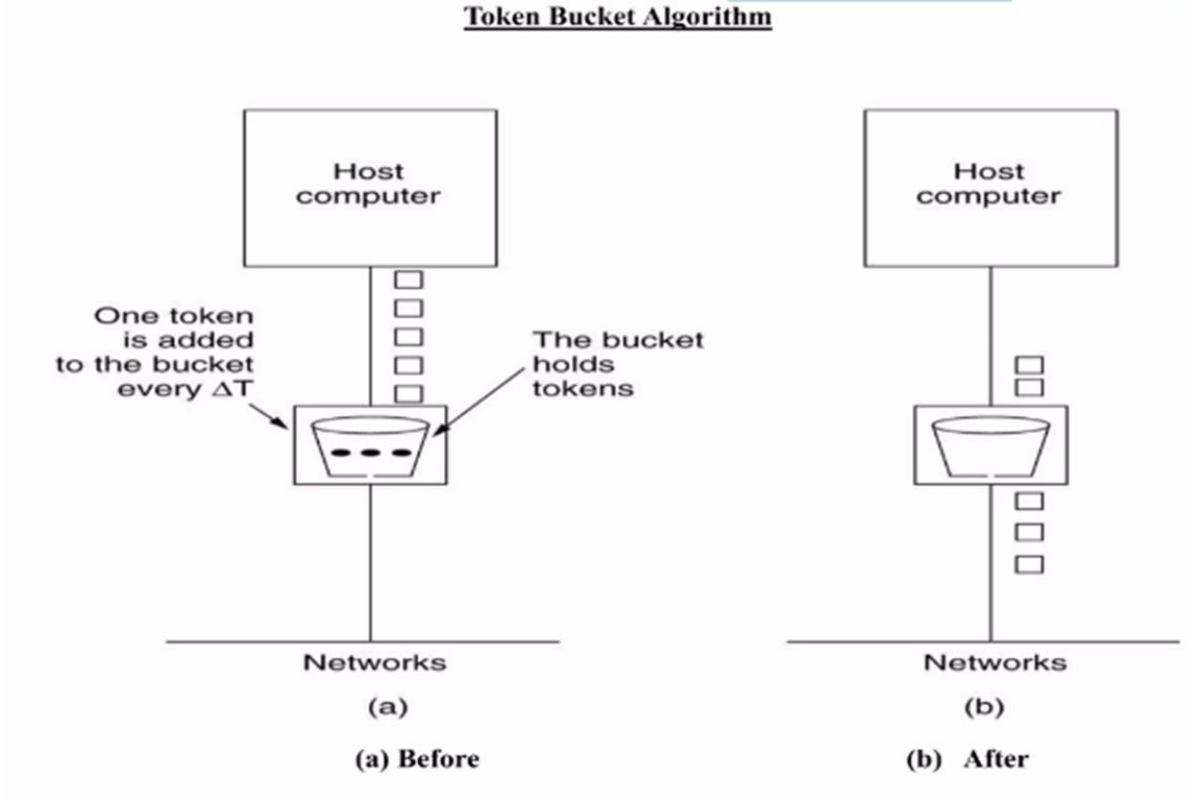
And we initialize **n = 1000** on another tick of clock.

This procedure is repeated until all the packets is sent to the network.

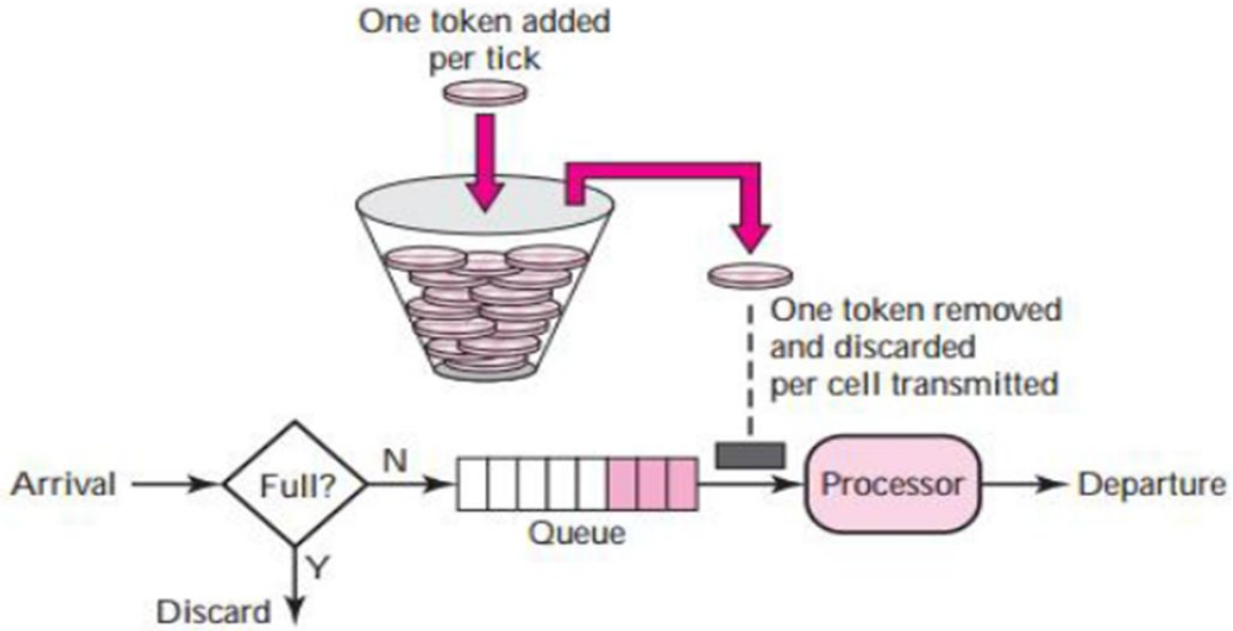
Token Bucket Algorithm:

- The token bucket algorithm allows idle hosts to save up permission to the **maximum size of bucket n** for burst traffic size.
- In this algorithm, the buckets **holds token** to transmit a packet, the host most **capture** and **destroy** one token.
- Tokens are generated by a clock at the rate of one token every change in time sec.
- Idle host can capture and save up token (up to the **max size of the bucket**) in order to send **large bursts later**.

Token Bucket Algorithm:



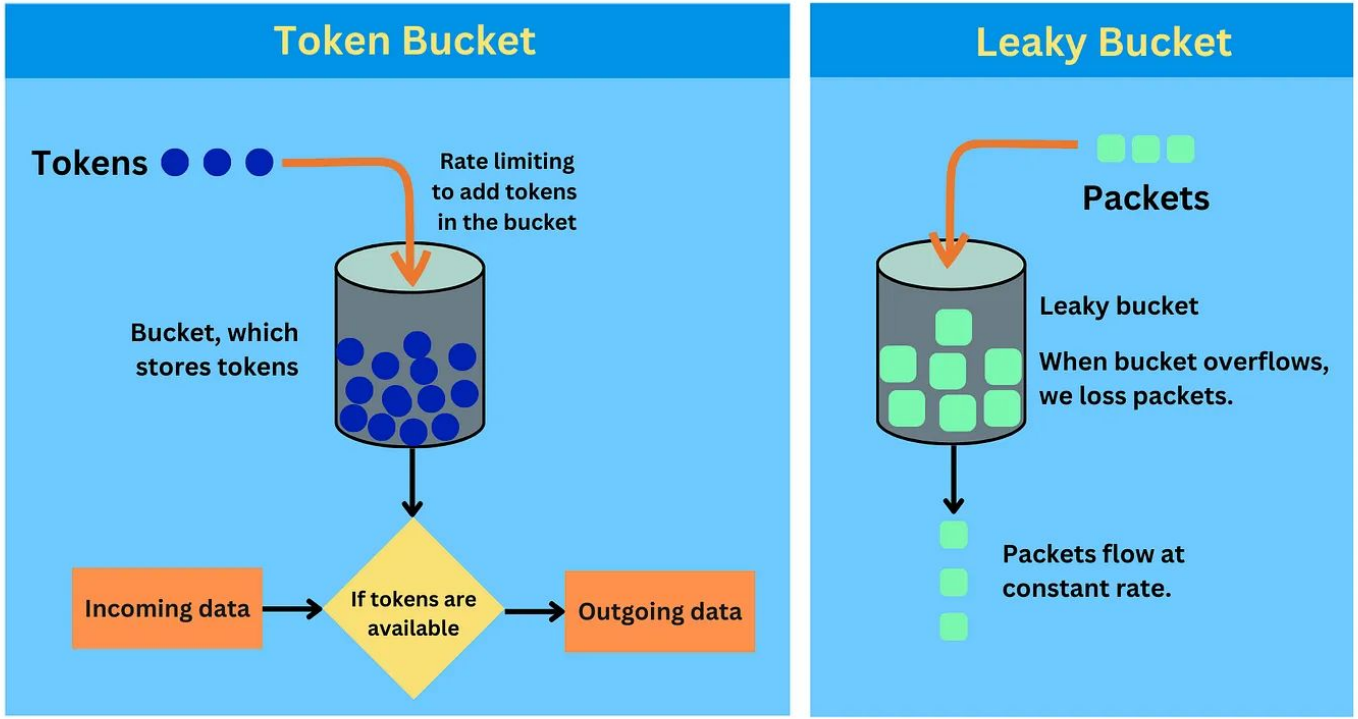
Token Bucket Algorithm:



Algorithm:

- **Step1:** A token is added at **everyΔt** time.
- **Step2:** The bucket can hold **at most b-tokens**. If a token arrive when **bucket is full** it is discarded.
- **Step 3:** When a **packet of m bytes a**rrived **m tokens are removed** from the bucket and the packet is sent to the network.
- **Step4:** If **less than n tokens** are available **no tokens** are removed from the bucket and the packet is considered to be **non conformant.**
- The **non-conformant packet** may be enqueued for **subsequent transmission** when token have been **accumulated** in the bucket.
- If c is maximum capacity of the bucket and p is the arrival rate and M is the maximum output rate then burst length S can be calculated as **S = C + P*S= MS**

Token Bucket vs Leaky Bucket:



Token Bucket vs Leaky Bucket:

Token Bucket Algorithm	Leaky Bucket Algorithm
It depends on tokens.	It does not depend on tokens.
If bucket is full, token is discarded but not the packet.	If bucket is full, then packets are discarded.
Packets can only transmit when there are enough tokens.	Packets are transmitted continuously.
Allows large bursts to be sent at faster rate. Bucket has maximum capacity.	Sends the packet at a constant rate.
The bucket holds tokens generated at regular intervals of time.	When the host has to send a packet , packet is thrown in bucket.
If there is a ready packet , a token is removed from Bucket and packet is send.	Bursty traffic is converted into uniform traffic by leaky bucket.
If there is no token in the bucket, then the packet cannot be sent.	In practice bucket is a finite queue outputs at finite rate.

Queuing Techniques for Scheduling

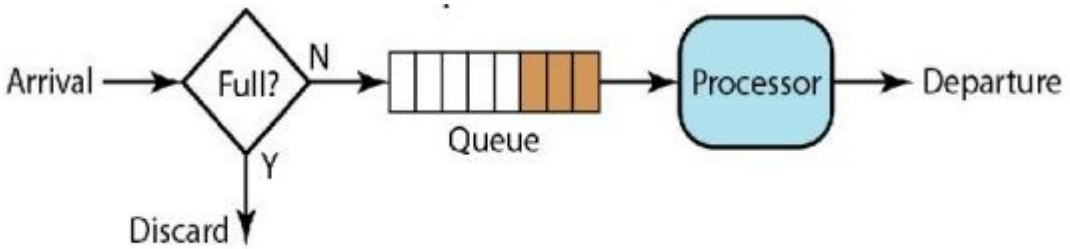
5.6

Queueing Techniques:

- QoS traffic scheduling is a **scheduling methodology** of network traffic based upon **QoS** (Quality of Service).
- Packets from **different flows** arrive at a switch or router for processing.
- A good scheduling technique treats the different flows in a **fair and appropriate** manner. Several scheduling techniques are designed to improve the quality of service.
- **Major scheduling techniques** are:
  - FIFO Queuing
  - Priority Queuing
  - Weight Fair Queuing.

FIFO Queuing:

- In **first- in first-out** (FIFO) queuing, packets **wait in a buffer** (queue) until the node (router or switch) is **ready to process** them.
- If the average **arrival rate** is higher than the **average processing rate**, the queue will fill up and new packets will be discarded.
- A FIFO queue is familiar to those who have had to wait for a bus at a bus stop.

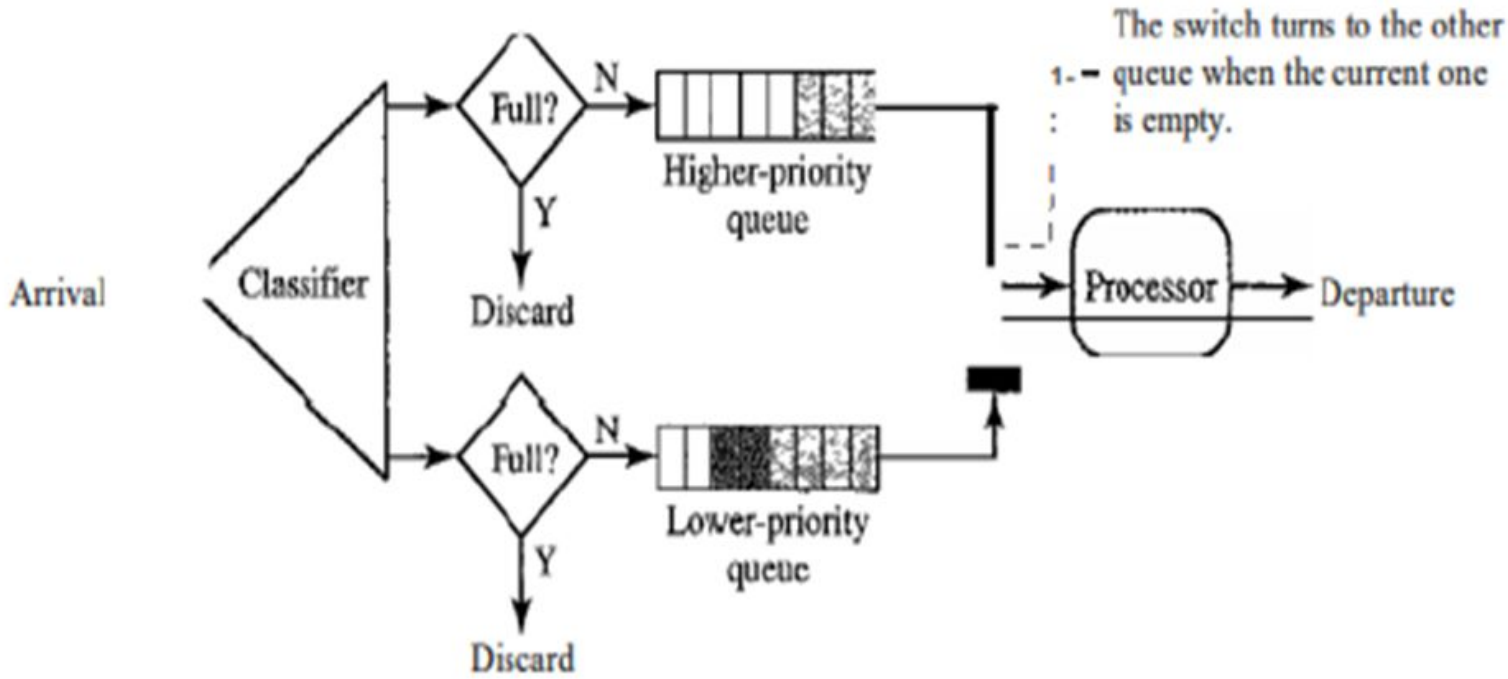


FIFO queue

Priority Queuing:

- In priority queuing, packets are **first assigned** to a **priority class.** Each priority class has its **own queue**.
- The packets in the **highest-priority queue** are processed first.
- Packets in the lowest-priority queue are **processed last**.
  - Note that the system **does not stop** serving a queue until it is empty.
- A priority queue can provide **better QoS** than the FIFO queue because higher priority traffic, such as multimedia, can reach the destination with **less delay**.
- However, there is a potential drawback. If there is a **continuous flow** in a high-priority queue, the packets in the **lower-priority queues** will **never** have a chance to be **processed**. This is a condition called **starvation.**

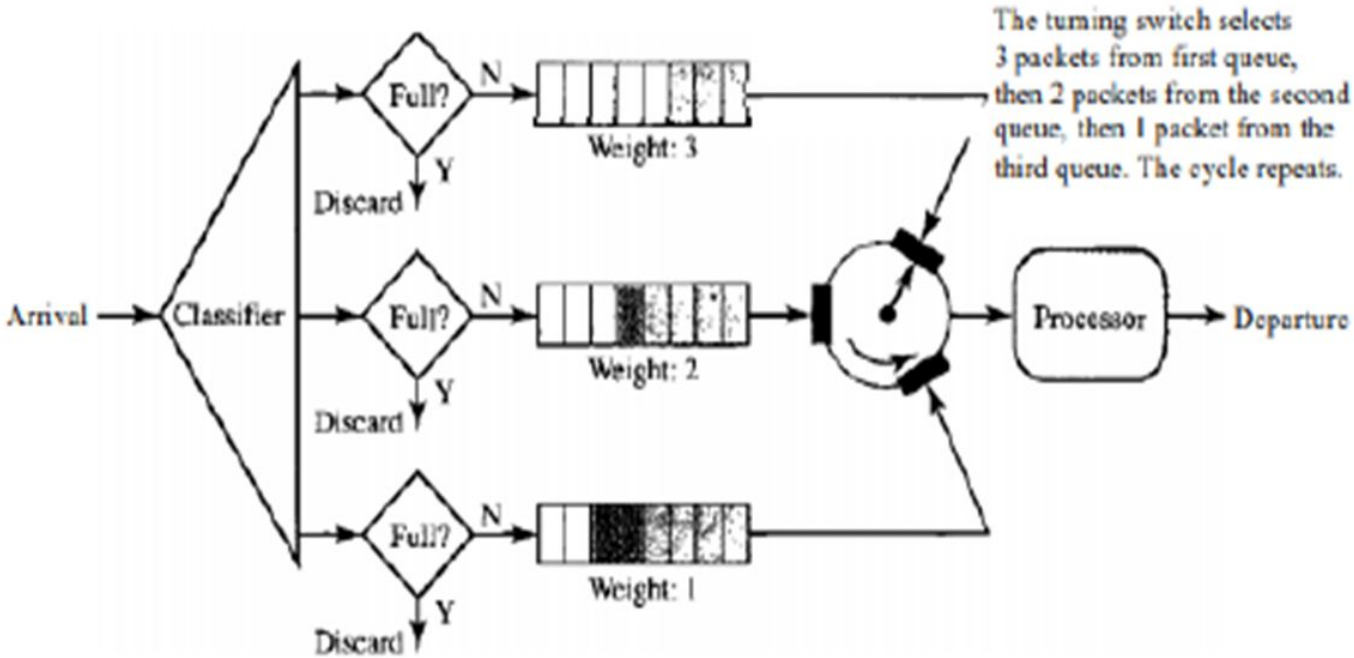
Priority Queuing:



Weighted Fair Queuing:

- A **better scheduling method** is weighted fair queuing. In this technique, the packets are still **assigned** to different classes and admitted to **different queues**.
- The queues, however, are **weighted** based on the **priority of the queues**; higher priority means a **higher weight.**
- The system **processes packets** in each queue in a **round-robin fashion** with the number of packets selected from each queue based on the c**orresponding weight**.
- For example, if the weights are 3, 2, and 1, **three packets** are processed from the **first** queue, **two from** the **second** queue, and **one** from the **third** queue. If the system does not impose priority on the classes, all weights can be equal. In this way, we have fair queuing with priority.

Weighted Fair Queuing:



Weighted fair queuing

Introduction to Ports and Sockets, Socket Programming

5.7

Overview:

- The **IP address** and the **physical address** are necessary for a quantity of **data to travel** from a source to the destination host.
- However, arrival at the destination host is **not the final objective** of data communications on the Internet.
- Today, computers are devices that can run multiple processes at the same time. The **end objective** of Internet communication is a **process communicating with another process**.
- For example, **computer A** can communicate with **computer C** by using **TELNET**. At the same time, computer A communicates with **computer B** by using the File Transfer Protocol (FTP). For these processes to receive data simultaneously, we need a **method to label the different processes.**

Overview:

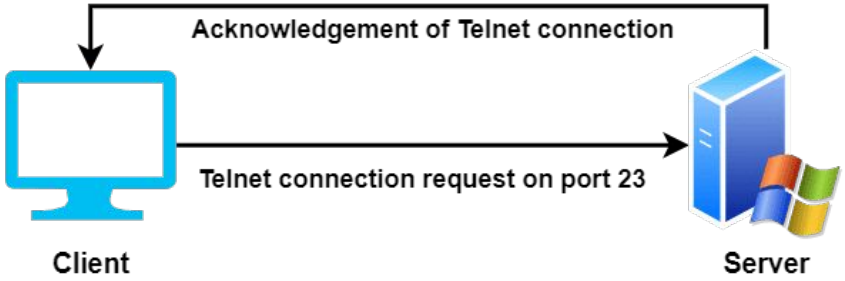
- In other words, they need addresses. In the TCP/IP architecture, the **label assigned** to a **process** is called a **port address**. A port address in TCP/IP is 16 bits in length.
- Source and destination addresses are found in the **IP packet**, belonging to the network layer.
- A transport layer datagram or segment that uses **port numbers** is wrapped into an **IP packet** and transported by it.
- The network layer uses the IP packet information to transport the packet across the **network** (routing). Arriving at the destination host, the **host's IP stack** uses the transport layer information **(port number)** to pass the information to the application.

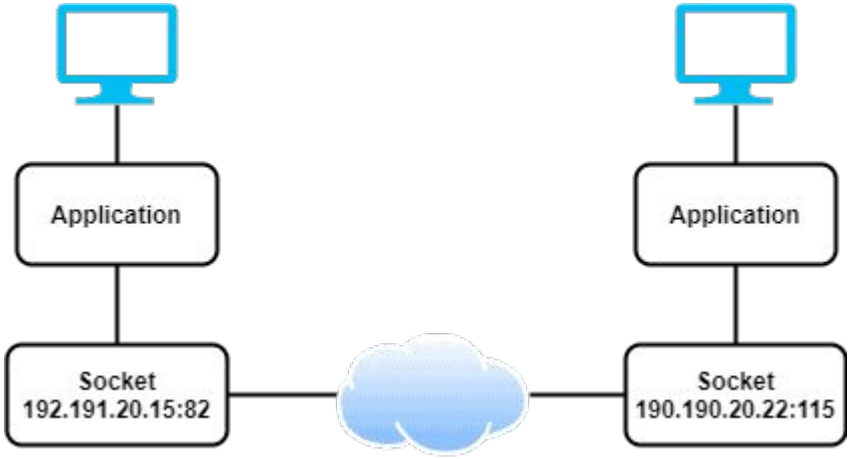
Overview:

- **IP address** and **Port Number** is combinedly called **Socket Address** that identifies the host along with the networking application running in the host.
- A socket is **one endpoint** of a two-way communication link between two programs running on the network.
- A socket is bound to a **port number** so that the TCP layer can identify the application that data is **destined to be sent** to.
- An endpoint is a **combination** of an IP address and a port number.

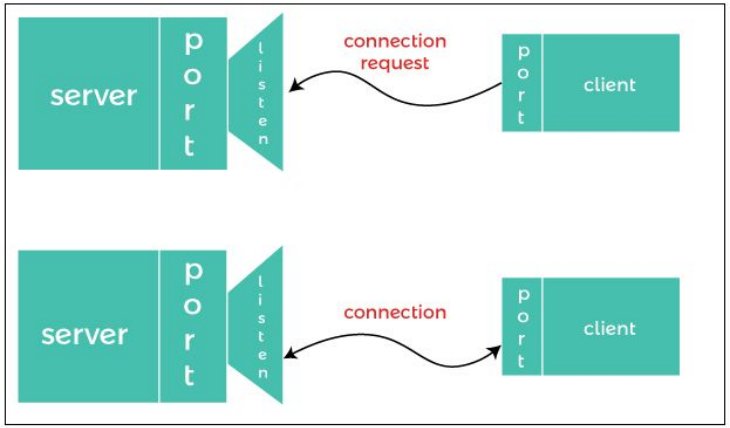
Port Number	Assignment
21	File Transfer Protocol (FTP)
23	TELNET remote login
25	SMTP
80	HTTP
53	DNS

Port and Socket:





NETWORK PORT	
Well-Known Ports	0 - 1023
Registered Ports	1024 - 4915
Dynamic Port	49152 - 65565



Socket Programming:

- A typical network application consists of a **pair of programs**
  - **a client program** and
  - **a server program**, residing in **two different** end systems.
- When these **two programs** are executed, a **client process** and a **server process** are **created,** and these processes communicate with each other by reading from, and writing to, sockets.
- When creating a network application, the developer's main task is therefore to write the **code for both** the client and server programs, called **socket programming**.

Socket Programming:

There are **two types** of network applications.

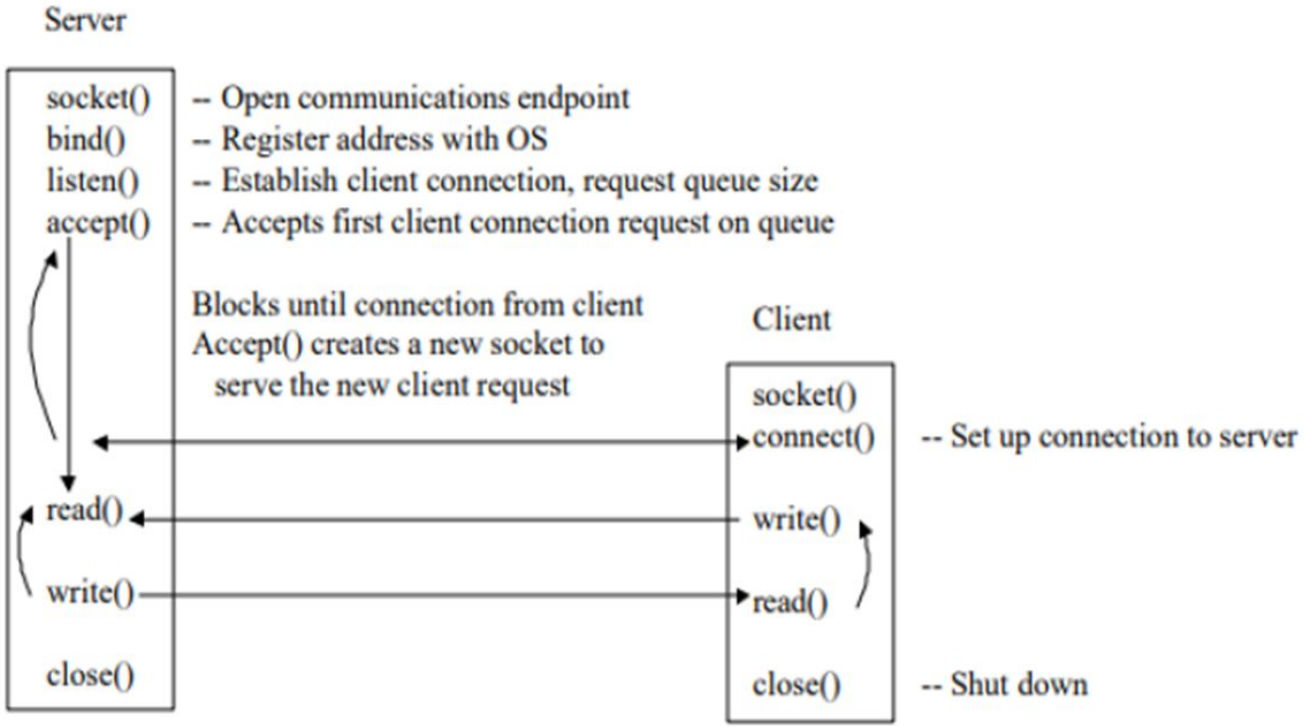
- **Implementation:** One type is an **implementation** whose operation is **specified in** a protocol standard, such as an **RFC** or some other standards document; such an application is sometimes referred to as **"open**," since the rules specifying its operation are **known to all**. For such an implementation, the client and server programs must **conform to the rules** dictated by the RFC.

- **Proprietary Network Application:** In this case the client and server **programs employ** an **application-layer protocol** that has **not been** openly published **in an RFC** or elsewhere. A single developer (or development team) creates **both the** client and server programs, and the developer has **complete control over** what goes in the code. But because the code does not implement an open protocol, other independent developers **will not be able** to develop code that interoperates with the application.

Socket Programming:

- When a **web page is opened**, automatically a **socket program** is **initialized** to receive/send to the process.
- The socket program at the **source communicates** with the socket program at the **destination machine** with the associated **source port/destination port** numbers.
- When a web page is **terminated**, automatically the socket programs will be **terminated**.

Socket Programming:



Socket Programming:

- The following sequence of events occur in the client-server application using **socket programming** for both **TCP and UDP:**
  - The client reads a **line of characters** (data) from its keyboard and **sends the data** to the server.
  - The server receives the data and converts the **characters to uppercase**.
  - The server sends the **modified data** to the client.
  - The client receives the modified data and **displays** the line on its screen.
- **BSD Socket API** is the well-known socket programming API that defines a set of standard function calls made available at the application level.
- **BSD-** Berkeley Software Distribution, **API-**Applications Programming Interface

Socket vs Port:

Parameters	Socket	Port
Definition	An endpoint for sending or receiving data across a network.	A numerical identifier for specific services or processes on a device.
Function	Facilitates communication between two devices.	Identifies different applications/services on a device.
Components	Consists of an IP address and a port number.	Consists solely of a number (0-65535).
Type	Exists in pairs (one on the client, one on the server).	Single numeric value.
Scope	Used for establishing and maintaining connections.	Used for routing data to the correct application.
Communication	Supports bidirectional data transmission.	Does not transmit data; helps direct data to sockets.
Protocols	Utilized in both TCP and UDP protocols	Defined within networking protocols (TCP, UDP, etc.).
State	Can be in different states (e.g., listening, established).	Does not have states.
Resource Usage	Consumes system resources (e.g., file descriptors).	Minimal resource usage, mainly memory.
Creation	Created by the operating system when a network application starts.	Predefined or dynamically assigned during a session.
Uniqueness	Unique combination of IP address and port number.	Only needs to be unique per IP address.

PYQs:

1. Explain TCP header with a neat diagram. Highlight on its uses.
2. Explain UCP header with a neat diagram. Highlight on its uses.
3. Why TCP is called a connection-oriented and reliable protocol? Differentiate TCP with UDP.
4. Why UDP is called a connection-less and unreliable protocol?
5. Explain various congestion control approaches.
6. What is open-loop congestion control? Compare it with closed-loop congestion control.
7. What is the difference between port and socket? Explain leaky bucket algorithm with example.
8. What is socket programming? Explain token bucket algorithm with an example.
9. Demonstrate the use of socket programming for creating network application using UDP and TCP with necessary diagrams.

THANKS!

Do you have any questions?

theciceerguy@p_m.me

9841893035

ghimires.com.np